

A.X K2 ALM (Audio Language Model)

SK Telecom

 <https://huggingface.co/skt/A.X-K2-ALM>

Abstract

This report presents A.X K2 ALM¹⁾ (Audio Language Model), a member of the A.X K2 family that adds a speech modality to the base LLM to enable natural voice-based conversation. A.X K2 ALM is built on A.X K2 Light²⁾, a 20B-A2.7B LLM, that is itself a down-scaled and partially modified variant of the A.X K2 architecture employing a Multi-head Latent Attention (MLA) design. Motivated by production settings—where the deployed LLM may be audio-unaware and its existing text-response quality must be preserved—we keep the LLM entirely frozen and train only the speech-side components, avoiding the catastrophic forgetting and text-quality degradation commonly induced by end-to-end multimodal training. Our model integrates speech understanding, voice activity detection (VAD), and speech generation within a single unified architecture built from four components: a chunk-wise Conformer speech encoder, an adapter combining an MLP projector and a Q-Former, a lightweight dual-head VAD module, and a multi-token-prediction (MTP) speech decoder. For understanding, the streaming Conformer encoder is self-supervised pre-trained on bilingual speech and then fine-tuned with an RNN-Transducer objective, and the adapter projects the resulting acoustic features into the LLM’s semantic space, with training stabilized by a student–teacher hidden-state alignment loss. For generation, the MTP decoder predicts hierarchical speech tokens at 12.5 Hz and renders waveforms through a neural audio codec decoder, yielding natural and low-latency synthesis. To support fluid turn-taking, the dual-head VAD module works in two stages—detecting speech onset from adapter outputs and predicting contextual offset from LLM hidden states—reducing false speech triggers from 16.5% to 9.2%. Evaluations on KVoiceBench, our Korean speech-understanding benchmark suite, demonstrate the effectiveness of our approach: A.X K2 ALM reaches 95% of the response quality of Qwen3-Omni while using only two-thirds of its parameters, and outperforms Qwen2.5-Omni and HyperCLOVA X 8B Omni by 21% and 29% relative, respectively, while achieving comparable response-speech quality. These results establish A.X K2 ALM as a scalable and computationally efficient blueprint for next-generation interactive voice agents. Moreover, because the backbone LLM remains frozen, the approach transfers readily to other LLMs.

1 Introduction

Speech is the most natural and intuitive interface for human-machine interaction. Building on the rapid advancement of large language models (LLMs) (Achiam et al., 2023; Comanici et al., 2025; SK Telecom, 2026), extending their multimodal capabilities to audio has emerged as a key research frontier. This push has led to the development of audio language models (ALMs) and omni-modal architectures capable of end-to-end speech understanding and generation (OpenAI, 2024; Xu et al., 2025; Qwen Team, 2025; Défossez et al., 2024; NAVER Cloud HyperCLOVA X Team, 2026; Kimi Team, 2025). Traditional voice interfaces typically rely on cascaded pipelines combining modular components: Voice Activity Detection (VAD), Automatic Speech Recognition (ASR), an LLM, and Text-to-Speech (TTS). While practical, these pipelines suffer from critical flaws: information transfer across module boundaries is strictly textual, discarding rich paralinguistic cues such as pitch and stress; errors cascade from upstream modules; and processing latency accumulates at each stage. ALMs address these limitations by directly mapping speech features into the LLM’s internal representation space, enabling unified speech understanding and response generation.

Existing approaches to coupling speech with LLMs generally fall into two paradigms. The first paradigm *jointly fine-tunes the LLM’s parameters* to introduce speech capabilities. This includes instruction-tuned speech understanding models like Qwen2-Audio (Chu et al., 2024) and SALMONN (Tang et al., 2024), as well as fully integrated omni-modal models such as Qwen2.5-Omni (Xu et al., 2025), Qwen3-Omni (Qwen Team, 2025), HyperCLOVA X 8B Omni (NAVER Cloud HyperCLOVA X Team, 2026), LLaMA-Omni (Fang

¹⁾To be released on Hugging Face soon.

²⁾A temporary, non-official model name used in this report, currently under development and validation.

et al., 2024), and Moshi (Défossez et al., 2024). Although these models demonstrate strong speech performance, fine-tuning the base LLM introduces severe trade-offs. Because spoken question-answering corpora are far smaller than text pre-training datasets, fine-tuning often triggers *catastrophic forgetting* of pre-trained textual knowledge, leaving a persistent performance gap between text and speech inputs on the same backbone (Wang et al., 2024). Furthermore, such instruction-tuned models frequently struggle to generalize to unseen tasks despite excelling on training distributions (Fan et al., 2025).

The second paradigm *keeps the LLM fully frozen* and connects speech input through lightweight, trainable adapters and representation alignment. Originating from vision-language designs like BLIP-2 (Li et al., 2023) and Flamingo (Alayrac et al., 2022), this approach was adapted to speech by BLSP (Wang et al., 2023) and AudioChatLlama (Fathullah et al., 2024), which train adapters to align speech representations with corresponding text transcripts, preserving the pre-trained LLM’s original capabilities. Subsequent works, such as Freeze-Omni (Wang et al., 2024), AlignFormer (Fan et al., 2025), and TASU (Peng et al., 2026), have further improved alignment quality and conversational capabilities, even demonstrating that frozen LLMs can capture subtle paralinguistic nuance (Kang et al., 2025). The core challenge of this paradigm lies in resolving the modality discrepancy caused by length mismatches between dense speech frames and discrete text tokens (Fan et al., 2025)—the fidelity of this cross-modal mapping directly dictates overall performance.

Our work follows the frozen-LLM paradigm, heavily driven by practical demands in production service settings. Industrial LLM deployments often rely on specialized or fine-tuned text backbones, many of which lack native audio support. Updating base LLM weights to add speech capabilities creates substantial risk, as it can degrade carefully tuned text quality and response behavior. In contrast, attaching a speech interface to a fully frozen backbone guarantees that existing text capabilities remain untouched, providing a low-risk, reusable adaptation recipe across diverse LLM backbones. Furthermore, current open omni-modal models are predominantly tailored to English and Chinese; their performance on Korean ASR and speech comprehension lags significantly, presenting a major bottleneck for deployment in Korean-speaking markets (NAVER Cloud HyperCLOVA X Team, 2026; KRAFTON AI, 2026).

To address these challenges, we present **A.X K2 ALM**, an audio language model built upon our in-house Mixture-of-Experts backbone, A.X K2 Light (20B total / 2.7B active parameters). Throughout our architecture, all LLM parameters remain frozen; only the speech-side modules are trained: a chunk-wise Conformer speech encoder (Gulati et al., 2020) trained from scratch on extensive English and Korean speech corpora, an adapter module (combining an MLP projector and a Q-Former) mapping acoustic features into the LLM token embedding space, lightweight context-aware VAD heads operating over internal representations, and a multi-token speech decoder that synthesizes spoken outputs from LLM representations. All streaming paths are optimized for low-latency voice interaction, with the total system scaling to 22B-A4B parameters.

We highlight the primary contributions of A.X K2 ALM as follows:

- **A Korean ALM Built Entirely from Scratch:** Every speech-side component—the streaming speech encoder, the adapter, the VAD heads, and the multi-token speech decoder with its neural audio codec—is built and trained in-house from scratch on top of the frozen A.X K2 Light backbone, rather than adapted from open-source speech models such as Whisper (Radford et al., 2023). This allows the frame rate, streaming behavior, and language coverage of every stage to be tuned end-to-end for Korean voice services—a degree of design freedom that repurposing pre-trained components does not afford.
- **Forgetting-Free Modality Extension:** We extend a fully frozen LLM to the speech domain via an encoder-adapter architecture that delegates audio understanding entirely to speech-side modules. Our two-stage alignment strategy—combining a syllable-CTC char2token bridge with cosine alignment, followed by frozen-LLM-in-the-loop dialogue training with self-distillation—preserves the base model’s core text capabilities without distortion and scales seamlessly to other models in the A.X family.
- **Korean-Optimized Streaming Speech Encoder:** We pre-train a 300M-parameter chunk-wise Conformer encoder from scratch on 400K hours of unlabeled Korean and English audio using multi-codebook BEST-RQ (Chiu et al., 2022; Baumann et al., 2025), and fine-tune it into a 341M streaming transducer (Graves, 2012). Despite operating under strict low-latency constraints and being half the size of competing audio encoders, it achieves lower Korean ASR error rates than Qwen3-Omni (Sec. 5.2).
- **Integrated Context-Aware VAD:** Replacing external VAD engines, we formulate VAD as lightweight six-class prediction heads attached directly to internal representations. An adapter-level head detects speech onset at minimal computational cost, while an LLM-level head continuously verifies utterance boundaries using conversational context—remaining active during model output generation to natively support barge-in interaction (Ma et al., 2024). This system significantly reduces premature

endpointing at mid-utterance pauses and spurious triggers on non-speech sounds such as back-channels, compared to conventional acoustic VADs (Silerio Team, 2021) (Sec. 5.3).

- **Efficient Streaming Speech Decoder:** We introduce a 0.88B multi-token speech decoder that integrates a text-speech language model with a depth-transformer to hierarchically generate semantic tokens and residual acoustic codes in real time. Operating at roughly half the parameter footprint of current state-of-the-art baselines (e.g., Qwen3-TTS 1.7B, CosyVoice3 1.5B), our model achieves superior pronunciation accuracy with the lowest character error rate (CER) while maintaining high speech intelligibility comparable to leading systems, ensuring precise delivery of generated text responses in real-time ALMs (Sec. 5.4).

In summary, A.X K2 ALM achieves a balanced trade-off among speech comprehension, generation fidelity, and dialogue quality relative to model scale. On KVoiceBench—our Korean spoken-query benchmark derived from VoiceBench (Chen et al., 2024) with human-reviewed localization and Korean-native professional-knowledge items from KMMLU (Son et al., 2024) (Appendix A)—A.X K2 ALM achieves 95% of the response quality of Qwen3-Omni at two-thirds of its size, while outperforming Qwen2.5-Omni and HyperCLOVA X 8B Omni by 21% and 29% (relative), respectively. These results establish an effective foundation for Korean voice services and a scalable blueprint for extending production LLMs to speech.

Organization. The remainder of this paper details the model architecture (Sec. 2), the training recipe (Sec. 3), the real-time serving system (Sec. 4), and experimental evaluations (Sec. 5).

2 Model Architecture

2.1 Overview

Figure 1 illustrates the overall architecture of A.X K2 ALM, which unifies speech understanding, voice activity detection (VAD), and speech generation within a single framework. The model comprises a speech encoder, an adapter (an MLP projector followed by a Q-Former), a frozen LLM backbone, a VAD module, and a speech decoder, all coupled through the representation space of the LLM. Table 1 summarizes the specifications of each component.

Input speech is encoded into frame-level features, and the adapter then compresses and projects them into token-level embeddings in the LLM’s embedding space. These embeddings are injected into the

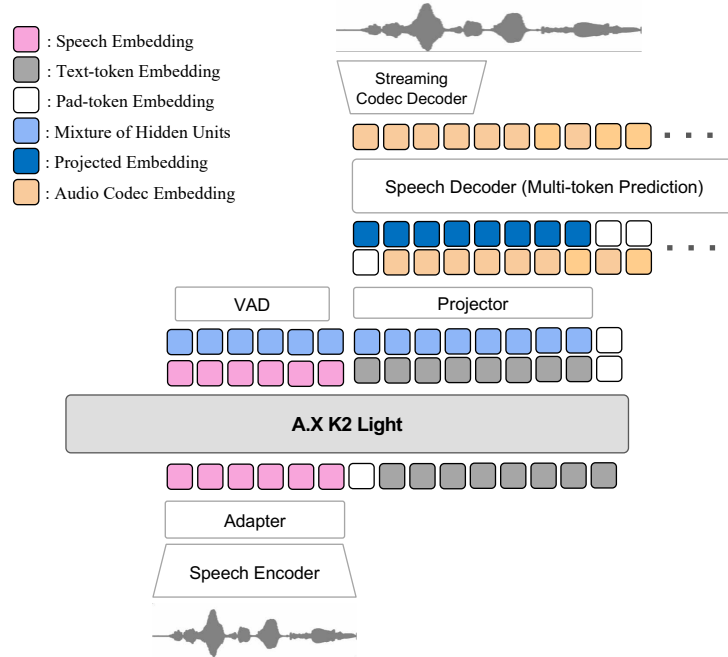


Figure 1: **Overall architecture of A.X K2 ALM.** Input speech is encoded by the Conformer speech encoder and projected into the LLM embedding space by the adapter (MLP projector + Q-Former); the frozen LLM performs understanding and response generation, and the speech decoder synthesizes the response back into speech. The VAD module (Sec. 2.5) additionally operate on the adapter outputs and LLM hidden states.

Table 1: **Component specification of A.X K2 ALM.** The backbone LLM is kept entirely frozen; all other modules are trained. Parameter counts of the lightweight VAD classification heads are negligible.

Function	Component	Configuration	Params
Speech Understanding	Speech Encoder	Chunk-wise Conformer	300M
	Adapter	MLP Projector (2-layer, GELU)	5.77M
		char-CTC head (linear, 2048 $\rightarrow$ 2667)	5.46M
	LLM (frozen)	Q-Former (4L / 16H / $d=2048$ / FFN 8192) A.X K2 Light (MoE)	201.4M 20B-A2.7B
Speech Generation	Conditioning Projector	FFN stack (2048 $\rightarrow$ 1024, 2 paths)	71.3M
	Multi-Token Predictor	Text-Speech Language Model	630M
	Speaker Encoder	Depth Transformer	110M
	Audio Codec Decoder	Transformer	100M
Total		A.X K2 ALM	$\approx$ 22B-A4B

LLM at a designated audio-placeholder position within the chat template, and the LLM generates a text response conditioned on them. The speech decoder then synthesizes speech directly from the hidden states the LLM produces while generating that response—using the emitted tokens only as an auxiliary content anchor rather than re-synthesizing from the decoded text. Voice activity is handled by a VAD module built from lightweight classification heads over these same internal representations rather than by an external front-end module, so every stage of the pipeline operates on a shared representation space.

Key design features:

- **Frozen LLM backbone:** the instruction-tuned LLM remains frozen throughout training, and audio embeddings are placed at a designated slot in the chat template it expects, with surrounding text tokenized and embedded following the model’s own conventions. This preserves the dialogue capability and response format of the underlying instruction model without modifying its weights.
- **End-to-end integration:** conventional spoken dialogue systems cascade VAD, ASR, an LLM, and TTS as independent modules, incurring information loss at every boundary. A.X K2 ALM instead maps speech directly into the LLM’s representation space for understanding, reads voice activity from classification heads over internal representations, and conditions the speech decoder on LLM hidden states rather than decoded text: coupling understanding, activity detection, and generation to a shared representation space and removing the module boundaries where information loss would otherwise occur.
- **Streaming by design:** the pipeline streams at both ends: the encoder and adapter process incoming audio incrementally as it arrives, and the LLM and speech decoder emit the response token by token rather than waiting for generation to complete — essential for low-latency live voice interaction. Because input is consumed incrementally, onset detection runs on the adapter output without occupying the LLM’s forward pass, letting the system keep listening even while a response is being generated.

2.2 Speech Encoder

The speech encoder is the core component that extracts frame-level, high-level representations from input speech, which serve as the speech-side interface for the adapter. We adopt a chunk-wise Conformer (Gulati et al., 2020)-a Transformer (Vaswani et al., 2017) augmented with convolution modules-to support streaming operation. The encoder takes 80-dimensional log-mel filterbank features, computed with a 20 ms window and a 10 ms shift, as input. Before entering the Conformer blocks, a 2-D convolutional front-end performs subsampling that lowers the temporal resolution, and further subsampling within the encoder brings the representation to a total frame period of 80 ms. This substantially reduces the subsequent self-attention cost — enabling efficient processing of long utterances — while compressing the local information of neighboring frames to refine the initial features. The encoder body stacks 24 Conformer blocks in the Macaron structure, in which a multi-head self-attention (MHSA) module and a convolution module are sandwiched between two feed-forward modules: MHSA models long-range dependencies within the utterance, while the convolution efficiently captures local correlations between neighboring frames. Layer normalization is used throughout, since normalization relying on batch statistics is unstable to estimate in a streaming setting. Table 2 summarizes the configuration.

For real-time output and recognition, the encoder operates over fixed-length chunks of 400 ms rather than the full utterance: MHSA is masked to attend only to the current chunk, a limited left context, and a limited

right context within the chunk. How much right context is allowed determines the trade-off between output latency and accuracy; to secure low latency, our design allows only the right context within the chunk. This chunk-wise context limitation is kept identical throughout pre-training and fine-tuning, minimizing the mismatch between training and deployment. The encoder comprises 300M parameters and is trained entirely from scratch on in-house and public Korean/English corpora (Sec. 3.1.1), rather than being initialized from an existing open-source encoder, allowing the frame rate, streaming behavior, and language coverage to be tailored to Korean-centric deployment. During ALM training, the encoder is updated in the speech-alignment stage (Stage A) and kept frozen in the speech-dialogue stage (Stage B), as described in Sec. 3.

2.3 Adapter

The adapter is the trainable module connecting the speech encoder and the frozen LLM (Li et al., 2023), consisting of an MLP projector and a Q-Former. The projector matches the dimensionality of the encoder features to the LLM embedding dimension, the Q-Former compresses frame-level representations into token-level representations, and the compressed representation is injected into the LLM after magnitude scaling. Since the LLM is fully frozen throughout, the responsibility of turning audio into a representation the frozen LLM can interpret lies with the speech-side components, in particular the adapter. The adapter data flow from the projector through the Q-Former to LLM injection is shown in Fig. 2.

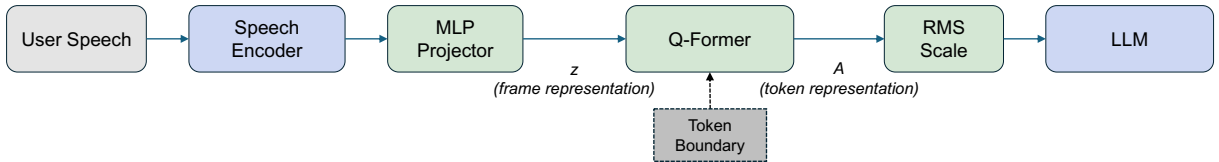


Figure 2: **Adapter data flow.** The MLP projector produces frame-level representations z ; the Q-Former summarizes the frames of each token window (injected as a boundary mask) into one token-level vector A ; per-token RMS scaling matches the magnitude of A to the LLM embedding distribution before injection.

MLP Projector. The projector is the first adapter component that maps the encoder’s output features into the embedding dimension used by the LLM. It normalizes the input with LayerNorm and passes it through a two-layer MLP (about 5.77M parameters) with a GELU activation in between. The output dimension equals the LLM embedding dimension (2048). We denote this frame-level output by z . The projector primarily mitigates the distributional gap between encoder features and language embeddings, matching only the dimension while preserving the frame resolution.

Q-Former. The Q-Former, adapted from the Querying Transformer of BLIP-2 (Li et al., 2023) (cf. the Perceiver Resampler of Flamingo (Alayrac et al., 2022)), is the core adapter component that compresses the variable-length frame-level representation z into token-level vectors. Whereas the projector only matches the dimension, the Q-Former defines, for each audio-aligned LLM token, a frame span (window) and summarizes the frames in that span into a single vector, thereby converting the temporal resolution into a token-level one.

Structurally, each layer consists of a pre-norm cross-attention followed by an FFN. In the cross-attention, the query is a single learnable query vector shared across windows, while the keys and values are the frame representations belonging to the window. Thus each token window produces exactly one output vector A . The window is injected as a mask indicating the frame-to-token assignment, built via

Table 2: **Speech encoder configuration.**

Item	Value
Input feature	80-dim log-mel filterbank (20 ms window / 10 ms shift)
Front-end	$2 \times \text{conv2d}$
Total subsampling / frame period	$8 \times$ / 80 ms
Number of Conformer blocks	24
Model dimension (d_{model})	768
Feed-forward dimension	2,560
Attention heads	8
Normalization	layer normalization
Chunk size	400 ms
Encoder parameters	300M

ground-truth forced alignment during training and a greedy path during inference (details in Sec. 3.1.2). The Q-Former has 4 layers, 16 heads, internal dimension 2048, and FFN size 8192, with one query per window (about 201.4M parameters).

LLM Injection and Output Scaling. During training, the token-level audio embedding A produced by the Q-Former is concatenated with the token embeddings of the system/user prompt and the teacher response into a single input sequence under teacher forcing, and the entire sequence is fed to the frozen LLM. During inference, A is placed at the audio-placeholder position of the chat template, after which the frozen LLM autoregressively generates the response.

Because A is trained with a cosine loss (constraining only direction, not magnitude; Sec. 3.1.2), injecting it directly may fall outside the magnitude range the LLM expects. To correct this, a per-token RMS normalization (Zhang & Sennrich, 2019) is applied just before injection: each token vector is divided by its own RMS and rescaled to the representative RMS of the LLM embeddings (`embed_rms`):

$$A_{\text{scaled}} = \frac{A_t}{\max(\text{rms}(A_t), 10^{-6})} \cdot \text{embed_rms}, \quad (1)$$

where `embed_rms` is a constant obtained as the per-token RMS average of the LLM embedding matrix. This scaling is applied identically during training and serving inference so that the audio embeddings share the same magnitude distribution as the text embeddings.

2.4 LLM Backbone

The backbone of A.X K2 ALM is A.X K2 Light, an in-house 20B-A2.7B (20B total, 2.7B active) instruction-tuned Mixture-of-Experts (MoE) model from the A.X family (SK Telecom, 2026). A.X K2 Light is a preview model — a down-scaled, partially modified variant of the A.X K2 architecture employing a Multi-head Latent Attention (MLA) design. The LLM interprets the user’s utterance, manages dialogue context, and produces the high-level semantic representations that drive response generation.

Crucially, all LLM parameters remain frozen throughout ALM training. This preserves the backbone’s language understanding and knowledge exactly, removes the need to retrain the full model when adding the speech modality, and minimizes the catastrophic forgetting that most omni-modal models suffer when their backbones are fine-tuned on multimodal data (Wang et al., 2024). The same extension recipe therefore applies unchanged to other A.X family models.

2.5 Voice Activity Detection (VAD)

In a spoken dialogue pipeline, a VAD at the front of the input crops out the speech segments so that only speech is passed to the encoder. Conventionally, this role is handled by an energy-based or general neural-network VAD (Silero Team, 2021). Because such a general VAD makes its decision based only on the acoustic signal, it tends to falsely detect segments that contain sound but are not user speech—such as music, multi-speaker babble, and environmental noise—as speech. Following prior work on frozen-LLM spoken dialogue (Wang et al., 2024), A.X K2 ALM removes the external VAD and redefines it as *lightweight classifiers placed on top of the contextual representations the pipeline already produces*: two lightweight heads map these representations to the six classes in Table 3. The **Adapter-based VAD** is a single linear layer with softmax over the adapter output, whereas the **LLM-based VAD** feeds the concatenation of the adapter output and the LLM hidden state through a small multi-layer perceptron (two hidden layers of 50 units each) followed by a softmax—so that the head combines the acoustic-level cue already available from the adapter with the contextual evidence carried in the LLM hidden state. A segment is regarded as speech if the softmax probability of class 0 (user speech) is the highest, and as non-speech otherwise. Going beyond a binary speech/non-speech decision, explicitly distinguishing the types of sound that are not speech is the core of this context-based decision making.

Table 3: The six classes defined in the VAD.

ID	Type	Description
0	Speech	User speech
1	Silence	Silence
2	Single-speaker voice noise	Voice noise from one person
3	Multi-speaker voice noise	Voice noise from two or more people
4	Music	Music noise
5	Other	Other noise

The two heads operate as a two-stage cascade in which the **Adapter-based VAD** detects the onset of speech and the **LLM-based VAD** performs verification and tracking. This division of roles provides two benefits:

- **Computational efficiency:** the Adapter-based VAD looks only at the adapter output, so it is cheap and fast but has limited context-based discriminative power; the LLM-based VAD looks at the LLM hidden state (concatenated with the adapter output), so it is highly discriminative but expensive to run at every step. When speech is not in progress, only the Adapter-based VAD runs to detect speech onset.
- **Speech detection during response generation:** by operating two separate VADs, the system can keep listening for speech even while the LLM is generating a response, enabling user barge-in (Ma et al., 2024).

Beyond this division into onset detection and verification, the two decisions are additionally combined while an utterance is being tracked—using the Adapter-based VAD’s judgment to refine the LLM-based VAD’s endpoint decision—which makes the resulting segmentation more robust to momentary fluctuations in the probability estimates.

2.6 Speech Decoder

The speech decoder converts the LLM’s output representations into audible speech, replacing the separate TTS invocation of cascaded systems. As shown in Fig. 3, it comprises a multi-token predictor followed by an audio codec decoder, operating on discrete speech tokens produced by a neural speech tokenizer (Fig. 3a). As illustrated in Fig. 3b, the Text-Speech Language Model predicts the coarse level-0 semantic token from a speaker embedding and text representations, and a Depth-Transformer predicts the residual acoustic codes at levels 1 through 15.

Speech Tokenizer. The tokenizer is the encoder and quantization stage of an auto-encoder-based neural audio codec that compresses 24 kHz speech into multi-group discrete tokens at 12.5 Hz (Défossez et al., 2022; 2024). A convolutional front-end and a Transformer encoder produce features, which are discretized into a semantic token by vector quantization (VQ) and into acoustic codes by residual vector quantization (RVQ). A codec decoder of Transformer blocks followed by a ConvNet then reconstructs the waveform, and the whole codec is trained on adversarial and reconstruction losses. The semantic codebook is aligned against a pretrained self-supervised speech model, WavLM (Chen et al., 2022), to concentrate on semantic content, whereas the acoustic codebooks capture detail such as timbre and prosody. The codec uses 16 codebook groups in total (1 semantic and 15 acoustic), and each codebook’s embedding table is reused to form the speech embeddings during multi-token speech decoder training.

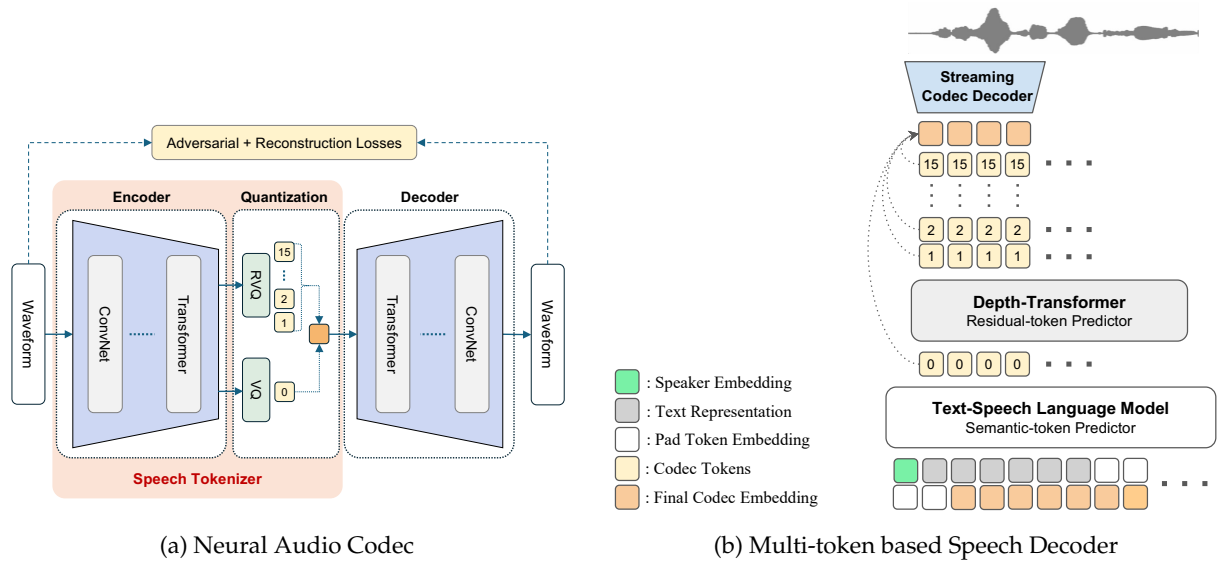


Figure 3: **Speech decoder.** (a) The Speech Tokenizer encodes the input waveform with a ConvNet-and-Transformer encoder, producing a semantic token via VQ and residual acoustic tokens via RVQ, and the Decoder reconstructs the waveform from them. (b) The Text-Speech Language Model predicts the coarse level-0 semantic token, and the Depth-Transformer predicts the residual acoustic codes (1–15).

Text-Speech Language Model. This module takes the text representations produced by the backbone LLM as input and predicts the semantic speech token for each frame. It is initialized from A.X-0.6B, one of our in-house language models, and then trained on text-speech paired data. On top of it, a linear head projects the final hidden state onto the codec vocabulary to predict the semantic speech token (codebook 0). The module retains the design of A.X-0.6B: grouped-query attention (16 query heads, 8 key/value heads), per-head query/key normalization, SwiGLU feed-forward blocks, and RMSNorm-based pre-normalization (Zhang & Sennrich, 2019). To synthesize speech matching the target speaker, it additionally conditions on a speaker embedding extracted from a 3-second reference utterance by a speaker encoder (Desplanques et al., 2020). The text representations shown in Fig. 3b are of two types:

- **TTS mode:** when the module runs as a standalone TTS model, the input text is tokenized and embedded, and these text-token embeddings are fed to the module.
- **A.X K2 ALM mode:** when integrated into A.X K2 ALM, both the text-token embeddings and the hidden states from the backbone LLM are jointly projected into the module’s input space. The projector is a pre-norm residual FFN stack: the input is linearly projected to the module’s dimension and refined by residual blocks, each applying RMSNorm, an expand-SiLU-contract feed-forward transform, and dropout, followed by a final RMSNorm. All linear layers and RMSNorm are bias-free. Two design choices enrich the conditioning. First, rather than tapping a single layer, the module fuses hidden states from all layers of the backbone LLM through learnable ELMo-style softmax weights (Peters et al., 2018), initialized to favor the last layer. Second, inspired by Qwen2.5-Omni’s Thinker-Talker conditioning (Xu et al., 2025), the same text-token embeddings are passed through a second projection of identical structure and added as a discrete content anchor.

Depth-Transformer. This shallow transformer predicts the acoustic speech tokens (codebooks 1–15) of each frame (Défossez et al., 2024). Rather than attending across frames, it operates on a per-frame depth sequence of length 15: the first position holds the embedding of the semantic token predicted for that frame, and each subsequent position holds the embedding of the preceding codebook within the same frame (intra-frame teacher forcing). Fifteen independent heads, one per acoustic codebook, read the output at each position to predict the corresponding token. Because no cross-frame dependency exists, frames are processed in fixed-size chunks, enabling efficient streaming inference.

Streaming Codec Decoder. For each frame, the embeddings of the 16 predicted tokens (one semantic and fifteen acoustic tokens) are summed to form a single frame representation. These per-frame representations are grouped into chunks of 8 frames (640 ms; 80 ms per frame) and converted into waveforms by the codec decoder (Transformer + ConvNet), enabling chunk-wise streaming synthesis in step with LLM output streaming.

Table 4 presents the detailed configuration of each speech decoder module described above.

3 Training

3.1 Speech Understanding (Encoder + Adapter)

The LLM stays fully frozen throughout training, so the task of making audio intelligible must be handled by the speech encoder and the adapter, not the LLM. The goal of training is therefore not to change the LLM itself but to convert audio into a form the LLM already understands. Training proceeds as follows:

- (1) **Speech encoder training** - self-supervised pre-training on 400K hours of unlabeled Korean/English speech, followed by RNN-Transducer ASR fine-tuning on 25K hours of labeled Korean speech (Sec. 3.1.1);
- (2) **Stage A (speech-text alignment)** - audio representations are aligned to the LLM token-embedding space so that the frozen LLM accepts audio as if it were text, via syllable-CTC boundaries, the char2token bridge, and cosine alignment (Sec. 3.1.2);
- (3) **Stage B (speech-text dialogue)** - on top of the aligned representation, the Q-Former is fine-tuned so that the frozen LLM reproduces, for audio input, the response ability it exhibits for text input, using front-weighted cross-entropy, logit distillation, and hidden-state alignment (Sec. 3.1.3).

Both adapter stages transfer the ability the frozen LLM already possesses onto the audio path rather than teaching it new knowledge. In Stage A, all components except the LLM (speech encoder, projector, char-CTC head, Q-Former) are trained; in Stage B, only the Q-Former is trained, while the front (speech encoder, projector, char-CTC head) and the LLM are frozen. The differences in what is trained versus frozen across the two stages are shown in Fig. 4.

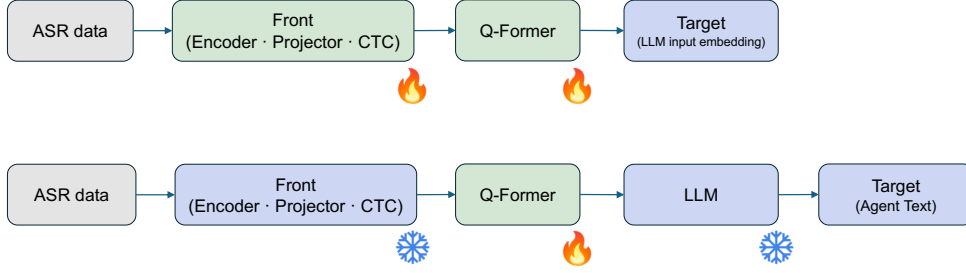


Figure 4: **Comparison of the Stage A (top) and Stage B (bottom) training structures.** Green with a flame denotes trainable modules; blue with a snowflake denotes frozen modules. Stage A trains the front (encoder, projector, char-CTC head) and the Q-Former toward LLM input embeddings as targets; Stage B trains only the Q-Former with the frozen LLM in the loop, supervised by teacher responses.

3.1.1 Speech Encoder Training

Self-supervised pre-training. To leverage large amounts of unlabeled speech, the encoder is pre-trained with BEST-RQ (Chiu et al., 2022), whose core idea is to generate training-target labels with a *random-projection quantizer that is fixed and never updated* throughout pre-training (Fig. 5). The quantizer consists of a randomly initialized, frozen projection matrix and codebook: input acoustic features are projected into a lower-dimensional space, and the index of the nearest codebook vector serves as the target for each frame. Because the codebook is fixed, no separate quantizer training or iterative clustering is required; random projection preserves, with high probability, the relative distance structure among high-dimensional points, so nearest-neighbor lookup after projection approximately reflects similarity in the original space. Thus, even though a fixed random projection carries no information by itself, forcing the encoder to consistently predict the targets at masked positions is sufficient for it to learn useful representations (Baevski et al., 2020; Hsu et al., 2021; Chen et al., 2022).

We use a multi-codebook configuration rather than a single codebook: several independent random projection-codebook pairs produce multiple targets per frame, and the encoder predicts them simultaneously. Predicting targets defined in different random subspaces enriches the self-supervised signal and reduces sensitivity to any particular quantization boundary, stabilizing representation learning (Baumann et al., 2025). Pre-training is framed as BERT-style masked prediction (Devlin et al., 2019): spans of input features are masked, a linear head on the encoder output produces a distribution per codebook, and the per-codebook cross-entropy loss is computed at masked positions. Pre-training is carried out under the

Table 4: **Speech decoder configuration.**

Category	Item	Value
Text-Speech LM	Layers	28
	Model dimension (d_{model})	1,024
	Feed-forward dimension	3,072
	Attention heads / KV heads	16 / 8 (GQA)
	Head dimension	128
	Activation / Normalization	SwiGLU / RMSNorm (with per-head QK-norm)
	Positional encoding	RoPE ($\theta = 500,000$)
	Max context length	32,768
Depth-Transformer	Layers	5
	Positional encoding	RoPE ($\theta = 10,000$)
	Prediction heads	15
Speech Tokenizer	Codebooks	VQ 1 (semantic) + RVQ 15 (acoustic)
	Codeword embedding dimension	256
	# of Codes	2,048
	Frame rate / Sampling frequency	12.5 Hz / 24 kHz
Conditioning Projector (A.X K2 Light → Text-Speech LM)	Source backbone (A.X K2 Light)	hidden size 2,048, 48 layers
	Input / output dim	2,048 / 1,024
	Structure	in_proj + 2 residual FFN blocks + RMSNorm
	Residual FFN hidden dim	8,192
	Projection paths	2 (layer-mixed hidden states + text-token anchor)
	Layer-mixing weights	49 (48 layers + embedding)
	Parameters	71,309,361 ($\approx 71.3\text{M}$)

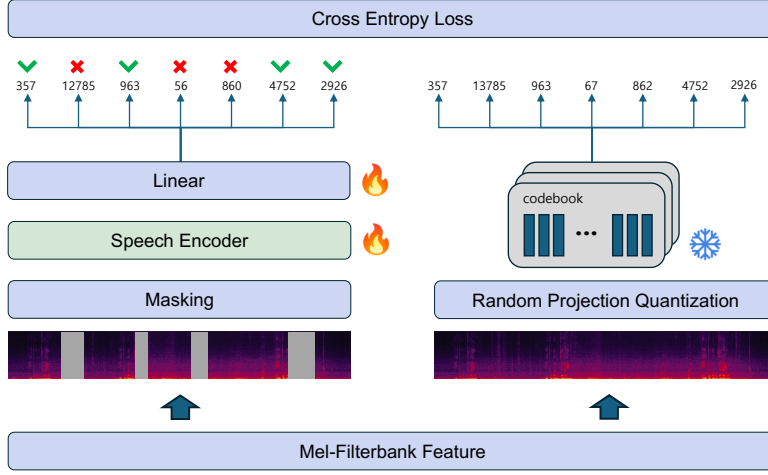


Figure 5: **Overview of masked-prediction pre-training with a frozen random-projection quantizer.** Masked log-mel features are encoded and a linear head predicts, at masked positions, the codeword indices produced by a frozen random-projection quantizer (right); flames denote trainable modules, and the codebook (snowflake) is frozen.

same chunk-wise streaming constraint as Sec. 2.2, so that representation statistics remain consistent with streaming recognition at fine-tuning. Table 5 summarizes the configuration. The training data comprise approximately 400K hours of unlabeled Korean and English speech drawn from in-house service logs, public corpora, and web-crawled data; training runs in a multi-GPU distributed setting with mixed precision, using AdamW with a cyclic learning-rate schedule for stable large-scale training.

Comparison with other self-supervised methods. Our choice of BEST-RQ over alternative self-supervised objectives is driven by streaming and scalability requirements:

- **Wav2Vec 2.0** (Baevski et al., 2020) uses a learnable Gumbel-softmax quantizer with a contrastive loss; the quantizer is trained jointly with the encoder, so the targets shift during training—disadvantageous for handling real-time input.
- **HuBERT** (Hsu et al., 2021) generates targets via offline k-means clustering and requires multiple rounds of iterative re-clustering, which is a burden at large data scale.
- **BEST-RQ** generates targets with a fixed random quantizer, requiring neither clustering nor quantizer training. This simplifies the pipeline and makes scaling to large-scale, multilingual pre-training straightforward; the multi-codebook configuration provides a wide target space designed to represent the fine acoustic variation of Korean speech.

ASR fine-tuning (Conformer Transducer). The pre-trained chunk-wise Conformer encoder is fine-tuned for the ASR task as an RNN-Transducer (RNN-T) (Graves, 2012), which naturally fits streaming recognition and, combined with the chunk-wise encoder, forms a low-latency streaming ASR system (Fig. 6). The Conformer Transducer consists of three sub-networks:

- **Encoder (transcription network):** the chunk-wise Conformer of Sec. 2.2, initialized from the pre-trained weights, encoding the acoustic input into frame-level representations;
- **Prediction network:** a label decoder over the emitted label history, consisting of a 512-dim label embedding followed by a 2-layer LSTM (hidden 1,536);

Table 5: **Self-supervised pre-training configuration.**

Item	Value
Number of codebooks	6
Codebook size	16,384
Codebook dimension	32
Masking probability	0.5
Mask span length	3
Minimum mask span	2
Prediction layer	linear (2 layers, 1,024 units, bottleneck 512)

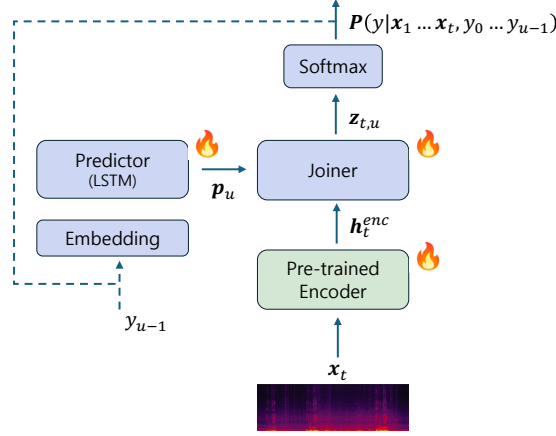


Figure 6: **Conformer Transducer architecture.** The pre-trained chunk-wise encoder (green), the LSTM prediction network, and the joint network are trained jointly with the RNN-T objective; flames denote trainable modules.

- **Joint network:** projects the encoder representation (768-dim) and the prediction-network output (1,536-dim) each into a 768-dim joint space, adds them, and computes a distribution over output tokens and the blank at each (time, label) position.

The output units are Korean wordpieces; including the blank and special tokens, the vocabulary size is 10,067, and the fine-tuned model totals about 341M parameters.

Fine-tuning minimizes the RNN-T loss, which marginalizes over the alignment latent variable; through blank emission, the Transducer learns a monotonic input - output alignment consistent with streaming decoding that consumes frames sequentially and emits partial results. A CTC loss (Graves et al., 2006), computed by a separate linear head on the encoder output, is used as an auxiliary objective, combined with equal weight (0.5 each; label smoothing 0.1): CTC directly enforces input-output monotonicity, helping the alignment converge early in training. The encoder is initialized from the pre-trained checkpoint while the predictor and joiner are randomly initialized; for the first 1,000 steps the encoder is not updated, so that the newly initialized decoder side stabilizes before joint training - preventing large gradients from the randomly initialized decoder from corrupting the pre-trained representation. Regularization applies dropout (encoder 0.1, decoder and embedding 0.4) together with SpecAugment (Park et al., 2019) (two frequency masks of max width 27, two time masks of max width 100). Fine-tuning uses approximately 25K hours of transcribed Korean speech covering service logs, public corpora, and crawled data, in a 40-GPU distributed setting with mixed precision, using AdamW with a OneCycle learning-rate schedule (peak 7×10^{-4}); a higher peak learning rate and shorter warm-up than in pre-training let the decoder converge quickly while preserving the pre-trained representation. Table 6 summarizes the main settings.

Table 6: **RNN-T fine-tuning configuration.**

Item	Value
Prediction network	2-layer LSTM (embedding 512 + hidden 1,536)
Joint network	768-dim
Vocabulary size	10,067 (Korean wordpiece)
Loss	RNN-T 0.5 + CTC 0.5 (label smoothing 0.1 each)
Dropout (enc / dec / emb)	0.1 / 0.4 / 0.4
Optimizer	AdamW ($\epsilon = 10^{-8}$, weight decay 0.01)
LR schedule	OneCycleLR (peak 7×10^{-4} , div_factor 25, pct_start 0.02, linear anneal)
Total parameters	341M

The resulting 341M-parameter streaming ASR system is evaluated in Sec. 5.2; the encoder is then carried into Stage A, where it is further updated, and frozen from Stage B onward.

3.1.2 Stage A (Speech-Text Alignment)

The purpose of Stage A is to align the audio embeddings produced by the Q-Former into the LLM’s token-embedding space so that the frozen LLM accepts audio as if it were text (Peng et al., 2026; Fan et al., 2025). Under the constraint that the LLM cannot be changed to fit audio, this alignment—moving

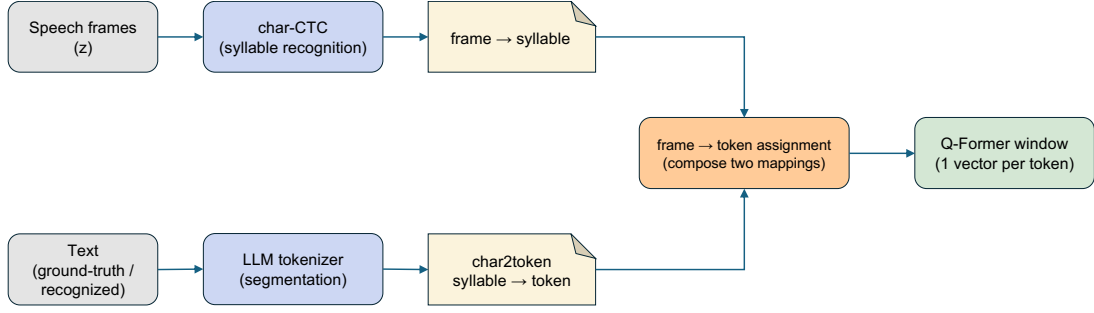


Figure 7: **The char2token bridge.** Frame-to-syllable boundaries from char-CTC are composed with the LLM tokenizer’s syllable-to-token segmentation to produce the frame-to-token assignment used as the Q-Former window (one vector per token).

audio toward representations the LLM already understands—is the adapter’s first objective, and it must precede meaningful response-generation training (Stage B). The goal is to make the single audio vector corresponding to each LLM token position point in the same direction as that token’s embedding, so that the resulting audio embedding lies closer to the corresponding text-token embedding from the LLM’s perspective.

Motivation for syllable-level alignment. To compress variable-length speech-frame representations into LLM token units, one needs a boundary (window) specifying which frame span corresponds to each token. The most direct approach is to learn frame-to-token boundaries directly by using the LLM’s subword tokens as CTC targets (Graves et al., 2006). However, because the subword vocabulary is large (tens of thousands) and context-dependent, short utterance spans suffer frequent misrecognition and the alignment tends to collapse onto a small number of tokens—a *windowing bottleneck* that degrades alignment quality and, in turn, the quality of the audio embeddings injected into the LLM. Noting that the syllable–speech alignment in Korean is relatively regular and clear, we introduce the *syllable* (character) as an intermediate alignment unit instead of the LLM token: we first secure clear frame–syllable boundaries via syllable-level CTC, then convert them to LLM token boundaries. The syllable vocabulary is small (thousands), giving dense learning signal and sharp boundaries, and the syllable-to-token conversion reuses the tokenizer’s deterministic segmentation, incurring no additional error. This structurally circumvents the windowing bottleneck of direct subword alignment.

Securing syllable boundaries via char-CTC. Syllable boundaries are obtained by attaching a syllable-classification head onto the projector output z (2048-dim). This head is a linear classifier ($2048 \rightarrow 2667$) distinguishing one blank token and 2,666 Korean syllables (about 5.46M parameters), trained with a CTC loss (Graves et al., 2006). Because its syllable inventory is defined solely for recognition/alignment, independent of the LLM vocabulary, this step focuses purely on the speech–syllable alignment problem. During training, forced alignment against the ground-truth syllable sequence assigns a syllable per frame. During inference, boundaries are decided by the CTC greedy path. The syllable CER converges from about 0.107 to 0.088, and the fraction of samples for which forced alignment is infeasible is only about 0.45%, confirming that the frame–syllable boundaries used downstream are stably formed for nearly all utterances.

char2token: converting syllable boundaries to token boundaries. Alignment hinges on composing two mappings to build a frame-to-LLM-token boundary. The first mapping, frame-to-syllable, is provided by char-CTC; the second, syllable-to-token (char2token), is obtained from the LLM tokenizer. Concretely, tokenizing the recognized or ground-truth syllable string with the LLM tokenizer reveals exactly which token each syllable belongs to, from the tokenizer’s segmentation information. Because subword segmentation is context-dependent, this mapping cannot be fixed as a global table and is computed dynamically for each sentence. Composing the two mappings determines which token each frame belongs to, and the Q-Former takes this assignment as its window to summarize each token span into a single vector. For example, a five-syllable Korean greeting may be recognized as five syllables but segmented by the LLM tokenizer into two tokens, giving $\text{char2token} = [0, 0, 1, 1, 1]$: the frames assigned to the first two syllables form the first token window, and those of the remaining three form the second. Because syllable CTC is densely trained over 2,666 classes (sharp boundaries) and the token segmentation uses the tokenizer’s actual output, the syllable-to-token conversion itself adds no further alignment error once the syllable sequence is fixed. The composition is illustrated in Fig. 7.

Training objective: char-CTC and cosine alignment. Stage A achieves two goals—recognition and alignment—simultaneously in a single training run. The speech encoder, projector, and char-CTC head are trained with a char-CTC loss so as to recognize syllables and form clear frame-syllable boundaries, while at the same time the token-level vector A produced by the Q-Former is trained with a cosine loss to point in the direction of the corresponding token’s LLM embedding. The two losses are not separated into distinct stages but are backpropagated together within one objective, so that boundary recognition and directional alignment are optimized jointly. The cosine loss is defined over valid token positions as the cosine distance between A and the ground-truth token embedding, and the overall objective adds the char-CTC loss to it as a weighted sum:

$$\mathcal{L}_{\cos} = \text{mean}(1 - \cos(A_t, E[\text{token}_t])), \quad \mathcal{L}_A = \mathcal{L}_{\text{CTC}} + \lambda \cdot \mathcal{L}_{\cos}, \quad (2)$$

where E is the LLM token embedding matrix. Because cosine similarity is invariant to magnitude, the cosine loss constrains only the direction of A and leaves the magnitude free. Hence, no separate magnitude normalization is applied at this stage, and matching to the LLM embedding magnitude is handled by the `embed_rms` scaling at injection time (Eq. 1). Training uses a large-scale speech-text corpus spanning diverse acoustic conditions to secure generalization across speakers, domains, and recording environments.

3.1.3 Stage B (Speech-Text Dialogue)

Stage B injects the audio embeddings aligned in Stage A into the frozen LLM and adapts the Q-Former using teacher answers as the training signal. Whereas Stage A optimizes proxy metrics such as cosine similarity, Stage B directly optimizes the final objective of response generation; it can be seen as fine-tuning the alignment from the perspective of response quality. The teacher signal is obtained from the same frozen LLM’s behavior when the same content is given as text, so the goal of Stage B is not to learn a new ability but to reproduce, for audio input, the response behavior already validated for text input. In this stage, only the Q-Former is trained, while the front (speech encoder, projector, char-CTC head) and the LLM are frozen—freezing the front also keeps the char-CTC-defined alignment windows consistent throughout dialogue training.

Objective and input construction. During training, the audio embedding A is produced by the same procedure as serving inference: greedy syllable recognition yields frame-syllable boundaries, `char2token` converts them to token boundaries, and the Q-Former produces token-level vectors. Because training and serving generate the audio embedding in the same way, the input representation used in training does not diverge from that in the actual service. The produced A passes through `embed_rms` scaling (Eq. 1), is inserted at the audio-placeholder position of the chat template, and is combined with the token embeddings of the system prompt and the teacher answer into a single input sequence of the form [system prompt | user turn with audio placeholder A | assistant (teacher answer)]. The loss is computed only over the teacher-answer tokens.

Training data and teacher signal. The purpose of the Stage B data is to make the frozen LLM interpret audio at the same level as text. To this end, each dialogue sample assigns, as the label for the audio input, a text response with the same content. Crucially, this reference response is generated not by an external model but by feeding the same content as text into *the same frozen LLM used for training*, decoded greedily without sampling. Meanwhile, user utterances (queries) are generated with a separate instruction-tuned LLM according to diverse scenarios and then converted into audio for use as input. This ensures the reference exactly matches what the LLM produces for text input, so the target the audio-input model must reproduce is consistently defined as the LLM’s own text-input behavior (self-distillation) (Wang et al., 2023; Fathullah et al., 2024).

The teacher signal used for knowledge distillation is also obtained offline during the same text-input pass and stored. Specifically, we store the top- k token log-probabilities at each response step and the hidden state just before the first answer token, using the former as the target for logit distillation and the latter for hidden-state alignment. Because both the reference answer and the teacher signal originate from the same frozen LLM’s text-input behavior, the teacher provides a text-input reference behavior that the audio path is trained to approximate.

Beyond this base data, we additionally build data targeting situations characteristic of the speech-dialogue environment. This stems from an input asymmetry inherent to this structure: as a multi-turn dialogue proceeds, user utterances accumulate in history as audio embeddings, whereas the agent’s previous answers accumulate as text tokens. The frozen LLM, being familiar with text, tends to over-attend to its own previously generated answers and thus prioritize its text answers over the newly arrived audio input. As a result, an erroneous answer can become difficult to correct in later turns, the model fixates on its previous answer and fails to follow topic changes, and information carried in past audio is missed.

To mitigate these failure modes, in addition to standard domains such as general knowledge and safety, we compose Korean multi-turn dialogues that include the following speech-dialogue-specific situations: inputs containing speech-recognition errors, topic transitions mid-dialogue, cases where the system prompt conflicts with the user request, barge-in where the user interrupts during the response, cases requiring reference to a previous turn’s audio, and cases where a text dialogue history is also provided. Among these, the past-audio-reference situation encourages the model to draw on not only the current utterance’s audio but also the past audio embeddings accumulated in history, while the recognition-error and topic-transition situations counteract the aforementioned tendency to fixate on its own text answers so the model learns to revise incorrect answers and follow topic changes in time.

Loss composition. When only the cross-entropy (CE) loss is used for response-generation training, the response tends to collapse to a limited set of high-frequency expressions early in training or become unstable, because in the frozen-LLM structure the trainable parameters are limited to the Q-Former and the adapter cannot sufficiently inherit the response ability the LLM already possesses. To mitigate this, CE is combined with two distillation losses:

$$\mathcal{L}_B = \mathcal{L}_{CE} + \lambda_{KD} \mathcal{L}_{KD} + \lambda_{HS} \mathcal{L}_{HS}. \quad (3)$$

The main loss $\mathcal{L}_{CE}$ is a next-token-prediction CE over the teacher-answer tokens. Since the beginning of a response often determines the overall direction and format, we apply *front-weighting* that varies the weight by token position—1.0 for the first 32 tokens, 0.5 up to the 96th token, and 0.25 thereafter—prioritizing the accuracy of the response’s opening.

The first distillation loss $\mathcal{L}_{KD}$ is logit knowledge distillation (Hinton et al., 2015), which minimizes the cross-entropy between the teacher’s top- k token distribution and the student’s log-probabilities so the student mimics it, conveying alternative-expression and confidence information not carried by a single ground-truth token; it uses the same front-weighting scheme as CE.

The second distillation loss $\mathcal{L}_{HS}$ is hidden-state alignment, which minimizes the cosine distance, at the last input position just before the first answer token, between the student’s hidden state and that of the teacher fed with text; this induces the input representation formed from audio to lie at the same point as the text-based representation, making the frozen LLM process audio input as if it were text.

The relative weights λ_{KD} and λ_{HS} are set to balance response stability against distillation strength. The three losses act complementarily: the front-weighted CE raises the accuracy of the response opening, logit distillation matches the output distribution, and hidden-state alignment matches the representation-space position to the text-input state, alleviating the early-response bias and instability seen with CE alone.

3.2 VAD Training

Unlike approaches that also train the underlying model (Radford et al., 2023), the encoder, adapter, and LLM are pre-trained and then kept frozen (Wang et al., 2024); the only components trained for VAD are the lightweight VAD heads over the six classes of Table 3—a single linear head for the Adapter-based VAD and a small multi-layer perceptron for the LLM-based VAD. Training proceeds in two steps:

- (1) **Adapter-based VAD:** since the LLM-based VAD verifies the Adapter-based VAD’s result at run time, the Adapter-based VAD is trained first. A speech chunk is passed through the encoder and adapter, and the linear head is trained to estimate the six classes on the resulting output.
- (2) **LLM-based VAD:** the trained Adapter-based VAD is run first, and once it detects the onset of speech, the adapter’s output from that point on is fed into the LLM—served with vLLM (Kwon et al., 2023)—to extract the corresponding hidden states. The LLM-based head, a small multi-layer perceptron (two hidden layers of 50 units each), is then trained on the *concatenation of the adapter output and the extracted LLM hidden state* to estimate the six classes; feeding the adapter output in parallel with the LLM hidden state lets the head combine the acoustic-level cue with the contextual evidence.

3.3 Speech Decoder Training

The training corpus is a large-scale, multi-speaker, bilingual (Korean/English) collection spanning read, conversational, broadcast, and in-the-wild speech, comprising approximately 39K hours of Korean and 88K hours of English. All audio is resampled to 24 kHz and tokenized with the pre-trained audio codec, yielding 16 codebooks at 12.5 Hz. Training proceeds in two phases: we first train the neural audio codec, then freeze it and train the multi-token predictor (Sec. 2.6) on the resulting tokens.

Neural Audio Codec. The codec is trained with a GAN-based reconstruction objective together with a vector-quantization (VQ) loss (Van Den Oord et al., 2017), following EnCodec (Défossez et al., 2022). The reconstruction term $\mathcal{L}_{\text{recon}}$ is an L1 loss over multi-scale mel-spectrograms. For adversarial training, we adopt a Multi-Resolution Discriminator (MRD) (Jang et al., 2021), which learns to distinguish the decoder’s reconstruction from the target speech, and we further apply a feature-matching loss $\mathcal{L}_{\text{feat}}$. The overall objective is

$$\mathcal{L}_{\text{codec}} = \mathcal{L}_{\text{recon}} + \lambda_{\text{vq}} \mathcal{L}_{\text{vq}} + \lambda_{\text{adv}} \mathcal{L}_{\text{adv}} + \lambda_{\text{feat}} \mathcal{L}_{\text{feat}}, \quad (4)$$

where $\mathcal{L}_{\text{adv}}$ is the MRD-based adversarial loss and each λ_* is the corresponding loss weight.

Multi-Token Predictor. The predictor is trained with next-token prediction and emits 16 tokens per frame: one semantic token and fifteen acoustic tokens. Its objective is a weighted sum of cross-entropy losses over the semantic and acoustic codebooks,

$$\mathcal{L} = \mathcal{L}_{\text{sem}} + w_{\text{code}} \cdot \mathcal{L}_{\text{code}}, \quad (5)$$

where $\mathcal{L}_{\text{sem}}$ is the semantic-token loss and $\mathcal{L}_{\text{code}}$ the acoustic-token loss. Since deeper RVQ codebooks incur larger losses and would otherwise dominate the gradient, we apply a depth-wise decay to the acoustic term,

$$\mathcal{L}_{\text{code}} = \frac{\sum_k \gamma^k \text{CE}_k}{\sum_k \gamma^k}, \quad 0 < \gamma \leq 1, \quad (6)$$

where k indexes codebook depth and CE_k is the cross-entropy of the k -th acoustic codebook. Setting $\gamma = 1$ recovers uniform weighting, whereas $\gamma < 1$ down-weights the deeper codebooks and thus mitigates their dominance over the gradient.

Training proceeds in two stages as follows:

- **(Stage 1) Pre-training.** This stage builds the multi-token predictor’s basic speech-generation ability. The Text-Speech Language Model learns the text-to-speech alignment on more than 100K hours of Korean and English speech, jointly with the Depth-Transformer and the linear heads. Two measures stabilize convergence. First, we train only the Text-Speech Language Model at the outset, then ramp the acoustic-loss weight w_{code} from 0 up to its target value. Second, to close the train–inference gap induced by teacher forcing, we replace a fraction of the acoustic-token inputs with random valid codes, making the model robust to error propagation at inference time.
- **(Stage 2) Fine-tuning.** Building on Stage 1, this stage refines synthesis quality on curated high-quality speech to achieve high-fidelity generation in the target speaker’s voice. To preserve the text-to-speech alignment learned in Stage 1, the Text-Speech Language Model is updated with a low learning rate (1×10^{-5}), whereas the speaker encoder and head layers use a higher rate (5×10^{-5}). We additionally introduce a speaker-classification objective based on Additive Angular Margin Softmax (Wang et al., 2018), which directly supervises the speaker embeddings so that they separate by speaker, thereby keeping the target-speaker characteristics consistent.

4 Serving System

The serving system consists of two main components: **ALM Control**, which orchestrates real-time client communications and asynchronous workflows, and the **Backends**, which execute model inference (Figure 8). The client connects to ALM Control via WebSocket to stream microphone audio and receive real-time server events. Incoming audio chunks are routed to the Encoder-with-VAD backend for voice activity detection and feature extraction. Concurrently, conversation contexts are dispatched to the Audio-LLM backend, which streams back synthesized text and audio. Within this pipeline, the proposed Speech Decoder (Sec. 2.6) serves as the core speech-generation engine.

4.1 ALM Control

ALM Control acts as the central orchestration server, handling bidirectional JSON-encoded event streams over WebSocket. It maintains a unified session context alongside four asynchronous tasks:

- **WebSocket Receiver / Sender:** The Receiver ingests streaming audio events from the client, while the Sender streams back server responses, including VAD status, incremental text (Δtext), and audio chunks (Δaudio).
- **Encoder-with-VAD Handler:** forwards incoming audio chunks to the Encoder-with-VAD backend and retrieves VAD states and speech embeddings.

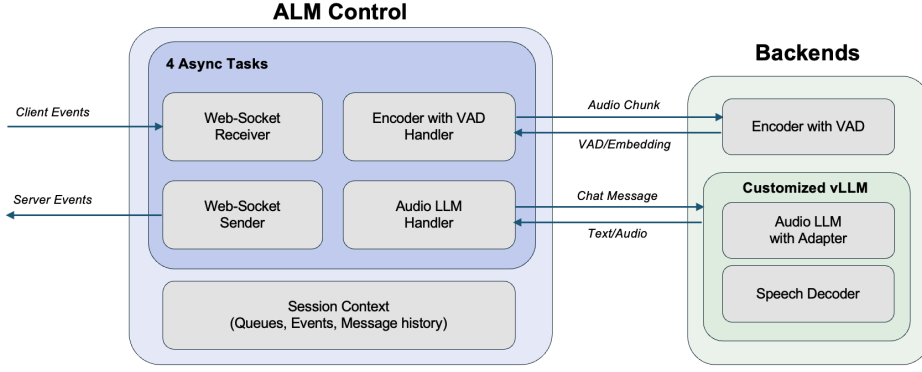


Figure 8: **Overview of the real-time ALM serving system.** ALM Control manages client interactions and session states via four asynchronous tasks along with a shared Session Context. The customized vLLM backend hosts an integrated pipeline comprising the Adapter, Audio LLM, and Speech Decoder.

- **Audio-LLM Handler:** dispatches conversation context messages to the Audio-LLM backend and handles the resulting text and audio streams.
- **Session Context:** manages inter-task queues, synchronization events, and conversation history to preserve session states.

Details regarding context management and request message schemas are provided in Appendix E.

4.2 Backends

The Backends consist of two primary inference components: the Encoder-with-VAD module and a customized vLLM engine. The Encoder-with-VAD module processes streaming audio chunks to extract speech embeddings while simultaneously determining Voice Activity Detection (VAD) states. The customized vLLM engine hosts an integrated pipeline comprising the Adapter, A.X K2 Light, and the Speech Decoder (Kwon et al., 2023). Within this pipeline, the Adapter first projects incoming speech embeddings into the LLM representation space. Next, A.X K2 Light generates text representations based on the conversation context. The Multi-token Predictor (Sec. 2.6) then converts these representations into multi-codebook speech tokens, which are ultimately reconstructed into raw audio waveforms by the Codec Decoder.

5 Evaluation

We evaluate A.X K2 ALM on four capabilities essential to a practical audio language model: speech recognition (STT), spoken question answering (Speech-QA), spoken instruction following (Speech-IF), and speech generation (TTS). Korean is our main deployment target, but current omni-model benchmarks offer limited coverage of it. We therefore construct KVoiceBench, a benchmark of Korean spoken queries that covers both Speech-QA and Speech-IF (Appendix A). Our comparison covers recent open-weight omni models: Qwen2.5-Omni-7B (Xu et al., 2025), Qwen3-Omni-30B-A3B (Qwen Team, 2025), and HyperCLOVA X 8B Omni (NAVER Cloud HyperCLOVA X Team, 2026) (released as HyperCLOVAX-SEED-Omni-8B).

5.1 Main Benchmark Results

Table 7 compares A.X K2 ALM with three recent open-weight omni models - Qwen2.5-Omni, Qwen3-Omni, and HyperCLOVA X 8B Omni - across four aspects of spoken interaction: speech recognition (STT), spoken question answering (Speech-QA), spoken instruction following (Speech-IF), and speech synthesis (TTS). These aspects cover perception, understanding, and generation, and together provide a broad view of end-to-end spoken-language capability. Across them, A.X K2 ALM remains competitive at a comparatively modest parameter scale.

Speech recognition. On the KsponSpeech eval-clean/eval-other splits, A.X K2 ALM records a CER of 9.00/9.12. This is lower than Qwen2.5-Omni (18.96/22.72) and HyperCLOVA X 8B Omni (10.22/10.15), and higher than Qwen3-Omni (8.46/7.91). For reference, the Stage-A-aligned variant that transcribes through the frozen LLM reaches 7.10/7.54 CER on the same splits, which is lower than any of the

Table 7: **Benchmark results of A.X K2 ALM against open-weight omni models.** OPEN denotes LLM-judged open-ended scoring (1–5 rescaled to 0–100); QA denotes answer accuracy. **Bold** indicates the best result. †: HyperCLOVA X 8B Omni (checkpoint HyperCLOVAX-SEED-Omni-8B) is evaluated with its default system prompt - its stronger configuration - whereas all other models use the common KVoiceBench prompt (Appendix A).

Task	Benchmark	Metric	A.X K2 ALM	Qwen2.5-Omni	Qwen3-Omni	HyperCLOVA X 8B Omni [†]
STT	KsponSpeech (eval-clean)	CER (%) ↓	9.00	18.96	8.46	10.22
	KsponSpeech (eval-other)	CER (%) ↓	9.12	22.72	7.91	10.15
Speech-QA (KVoiceBench)	AdvBench	OPEN ↑	92.67	92.61	89.55	81.87
	AlpacaEval	OPEN ↑	69.70	52.87	71.14	51.81
	AlpacaEval-Full	OPEN ↑	70.75	52.45	72.21	52.43
	BBH	QA ↑	44.18	52.66	57.22	43.80
	CommonEval	OPEN ↑	70.20	55.41	68.25	55.49
	MMSU	QA ↑	43.67	32.08	43.48	32.08
	OpenBookQA	QA ↑	76.62	76.12	91.54	70.90
	SD-QA	QA ↑	46.87	25.89	48.23	27.52
	WildVoice	OPEN ↑	66.88	46.33	66.25	45.77
Speech-IF	IFEval	OPEN ↑	61.20	44.88	67.61	35.21
Average (Speech-QA/IF)		↑	64.27	53.13	67.55	49.69
Speech Generation	KVoiceBench (Subset-200)	CER (%) ↓	1.86	–	26.37	4.16
		UTMOS ↑	2.871	–	3.178	3.171
		STOI ↑	0.990	–	0.999	0.996

models compared here. The difference between this variant and the final model reflects a trade-off introduced during speech-dialogue training (Stage B), in which improvements in conversational ability are accompanied by a reduction in transcription accuracy. Reducing this gap without degrading dialogue quality remains an open direction.

Speech understanding and instruction following. Averaged over the ten KVoiceBench subsets, A.X K2 ALM scores 64.27, corresponding to 95% of the average obtained by Qwen3-Omni (67.55), the highest-scoring baseline in this comparison, while using approximately two-thirds of its parameters. Relative to the remaining baselines, A.X K2 ALM exceeds Qwen2.5-Omni and HyperCLOVA X 8B Omni by 21% and 29% in relative terms. At the subset level, it obtains the highest score on AdvBench, CommonEval, MMSU, and WildVoice, and remains close to the leading model on both AlpacaEval variants. The remaining gap to Qwen3-Omni is concentrated in the knowledge-heavy multiple-choice subsets (BBH and OpenBookQA). We attribute this primarily to differences in model scale and in exposure to multiple-choice formats during training, rather than to the quality of the speech interface.

Speech generation. On KVoiceBench Subset-200, A.X K2 ALM shows the best pronunciation accuracy. Its character error rate (CER) is 1.86%, which is much better than HyperCLOVA X 8B Omni (4.16%) and Qwen3-Omni (26.37%). As a result, its generated speech expresses the intended text responses much more accurately than the baseline models. Speech clarity is similar to other systems, with a STOI of 0.990 compared to 0.996 and 0.999. However, overall sound quality is the main area where A.X K2 ALM falls behind, scoring a UTMOS of 2.871 against 3.178 for Qwen3-Omni and 3.171 for HyperCLOVA X 8B Omni. This gap comes mainly from the limited reconstruction quality of our neural audio codec, which upper-bounds the sound quality of the synthesized speech. Upgrading the codec is left to future work.

Overall, these results indicate that A.X K2 ALM attains competitive speech understanding and generation, together with dialogue quality that is maintained at a moderate model size, suggesting its suitability as a basis for deployment.

5.2 ASR Performance of the Speech Encoder

Table 8 reports Korean ASR error rates of our streaming ASR system (self-supervised pre-training + RNN-T fine-tuning, Sec. 3.1.1) against Qwen3-Omni, a publicly available state-of-the-art omni model. Evaluation is performed under the same chunk-wise streaming constraint used in training, allowing only the right context within a chunk. Our model achieves a lower CER across the clean and other conditions of KsponSpeech and on the full Zeroth-Korean set, while its model size (about 341M) is roughly half that of the comparison model’s audio encoder (600M). Given that these results were measured under a low-latency constraint—unlike a non-streaming model that can attend to the entire utterance—they show that the model attains competitive recognition performance even under real-time conditions, indicating

that from-scratch training on large-scale Korean-centric audio yields a stronger foundation for Korean speech understanding than adapting encoders optimized primarily for English and Chinese.

These results can be seen as the combined effect of the design choices described in Secs. 2.2 and 3.1.1: the Conformer captures both global and local context by combining MHSA and convolution; the chunk-wise structure and layer normalization enable low-latency streaming; multi-codebook masked-prediction pre-training learns representations at scale from unlabeled speech through a fixed random-projection quantizer; and fine-tuning—a Transducer on top of the pre-trained encoder with CTC as an auxiliary objective—connects these representations to real-time recognition performance. This combination forms the basis on which the speech encoder supports robust ALM performance in real time.

Table 8: **Korean ASR results: CER(%)↓**. Our model (self-supervised pre-training + RNN-T fine-tuning) is evaluated under the chunk-wise streaming constraint. **Bold** indicates the best result.

Test set	Ours (341M)	Qwen3-Omni (600M)
KsponSpeech (eval-clean)	6.13	8.46
KsponSpeech (eval-other)	6.35	7.91
Zeroth-Korean	1.86	3.33

5.3 VAD Results

Evaluation sets. We evaluate on three test cases (TCs). The first reflects realistic everyday usage, while the other two isolate specific phenomena that require semantic judgment beyond a simple speech/non-speech decision—each a case that a typical acoustic-based VAD tends to misjudge. Because natural recordings that isolate the pause and back-channel phenomena in sufficient quantity are hard to collect, the latter two sets were synthesized with TTS (ElevenLabs, 2025) to secure controlled speech patterns matching our objectives, whereas the first consists of real recordings:

- **General Recording TC** (29,395 ways): general utterances recorded in a variety of acoustic environments, from clean conditions to diverse background noise; some recordings also contain mid-utterance pauses or back-channel voice, so the set reflects realistic everyday usage rather than any single targeted phenomenon.
- **Pause TC** (1,200 ways): utterances in which a short silent segment naturally occurs while the speaker continues speaking. The goal is to maintain the utterance as a single ongoing segment across the pause, whereas an energy-based VAD tends to misjudge the pause as the end of the utterance and fragment it.
- **Back Channel TC** (1,200 ways): user speech overlaid with back-channel voice—the conversation partner’s acknowledgments or reactive vocalizations, which are present as sound but are not user speech. The goal is to exclude such sound from the speech segment, whereas an acoustic-based VAD tends to falsely detect any segment with meaningful acoustic energy as speech.

Metrics. We evaluate at the segment level whether the actual speech segment is captured intact as a single segment, and how many unnecessary segments are produced—capturing fragmentation, merging, and over-extension that boundary accuracy alone does not reveal. From the recall perspective (GT coverage): *mean coverage* (average ratio by which each ground-truth segment is covered by the union of predictions), *fully covered* (ground-truth segments whose coverage exceeds a threshold), *detected / missed* (at least partially captured / not captured at all), and *fragmentation* (average number of predicted segments covering a single ground-truth segment; 1.0 means one segment per utterance). From the precision perspective (false speech): *spurious* (predictions with almost no overlap with any ground-truth segment; count and total duration) and *over-extension* (predictions that sufficiently overlap a ground-truth segment but extend beyond it at the front or back by more than a threshold; if the overflow drops the overlap ratio below 0.5, the segment is classified as spurious). In addition, we report the frame-level binary speech/non-speech confusion—true and false positives and negatives (TP/FP/FN/TN), with speech as the positive class—and, for our own VAD heads, the six-class accuracy of Table 3, which a purely acoustic VAD cannot produce.

Setup. We report three systems. The **Adapter-based VAD** uses only the adapter-level head; the **LLM-based VAD** is the full two-stage cascade (Sec. 2.5), in which the Adapter-based VAD detects onset and the LLM-based head verifies and tracks the utterance with contextual evidence; and **Silero VAD** (Silero Team, 2021) is a purely acoustic baseline. Comparing the Adapter-based VAD against the cascade isolates the contribution of the LLM-based verification, while comparing the cascade against Silero contrasts a

context-aware decision with a purely acoustic one. Both of our systems make chunk-level decisions and Silero makes frame-level decisions; the state-machine control parameters are omitted here.

General Recording TC. On realistic recordings all three systems behave similarly at the coarse binary level (Table 9), covering essentially all speech (TP around 95%) with comparable miss rates; the differences appear in the false-speech and class metrics. Relative to the Adapter-based VAD, the cascade lowers the false-positive rate (13.7%→11.8%) and roughly halves the spurious segments (1,515→715), confirming that the LLM-based verification removes non-speech that the adapter head alone accepts. The six-class accuracies (Table 10) show the division of labor: the adapter head provides broad—if coarse—discrimination across the non-speech taxonomy, whereas the LLM-based head, invoked only on chunks near speech (about 48%), is extremely sharp on the speech class (99.2%) because its role is speech verification rather than fine non-speech typing. Relative to Silero, the cascade emits far less spurious speech in total duration (1,099 s vs. 3,489 s) and much less over-extension (628 s vs. 1,414 s); Silero reaches a higher fully-covered rate by cutting boundaries tightly, at the cost of these false detections.

Table 9: **General Recording TC (Adapter-based VAD vs. LLM-based VAD vs. Silero VAD).** TP/FN are over ground-truth speech frames, FP/TN over non-speech frames.

Metric	Adapter-based	LLM-based	Silero
GT segments	30,410	30,410	30,410
TP	95.3%	94.8%	94.9%
FN	4.7%	5.2%	5.1%
FP	13.7%	11.8%	13.3%
TN	86.3%	88.2%	86.7%
mean coverage	95.81%	95.20%	94.59%
fully covered (≥ 0.97)	75.0%	74.1%	83.4%
detected / missed	30,055 / 355	29,899 / 511	29,341 / 1,069
fragmentation (pred/GT)	1.05	1.04	1.02
pred segments	32,974	31,888	31,613
spurious (count / time)	1,515 / 2,337 s	715 / 1,099 s	1,382 / 3,489 s
over-extension (count / time)	1,175 / 806 s	940 / 628 s	1,809 / 1,414 s

Table 10: **Six-class accuracy on General Recording TC (Adapter-based vs. LLM-based head).** The LLM-based head is evaluated only on chunks near speech, where it is invoked (about 48% of chunks); Silero cannot produce a six-class decision.

ID	Class	Adapter-based	LLM-based
0	Speech	97.9%	99.2%
1	Silence	53.4%	13.5%
2	Single-speaker voice	23.1%	8.1%
3	Multi-speaker voice	39.6%	22.4%
4	Music	0.0%	0.0%
5	Other	38.0%	4.8%

Pause TC—maintaining speech across pauses. The key metrics are fragmentation and fully covered (Table 11). Relative to the Adapter-based VAD, the cascade sharply improves both—fully covered rises from 19.5% to 47.4% and fragmentation falls from 2.00 to 1.66—while cutting the false-positive rate (19.6%→8.0%) and reducing over-extension by an order of magnitude (1,252 sides / 1,151 s → 111 / 77 s). The Adapter-based VAD’s high false positives here stem largely from the boundary characteristics of pause-containing speech: such utterances are often not cleanly delimited—they tend to begin with hesitation fillers and to trail off as if the speaker will continue—so an adapter-level decision absorbs the surrounding sound as speech, whereas the LLM-based verification suppresses much of it using context. Relative to Silero the contrast is starker: Silero misjudges pauses as endpoints and fragments utterances (fully covered 0.8%, fragmentation 2.04), whereas the cascade keeps them intact across the pause. The cascade over-extends more than Silero for the same boundary reason: because these utterances tend to trail off rather than stop crisply, a VAD deliberately biased against premature termination pushes the endpoint slightly past the ground truth, whereas Silero’s tight silence cutting avoids over-extension but fragments the utterance instead. This over-extension trades off against the cascade’s far lower fragmentation and can be tuned via the state-machine parameters.

Table 11: **Pause TC (Adapter-based VAD vs. LLM-based VAD vs. Silero VAD).**

Metric	Adapter-based	LLM-based	Silero
GT segments	1,200	1,200	1,200
TP	91.9%	94.8%	86.8%
FN	8.1%	5.2%	13.2%
FP	19.6%	8.0%	3.0%
TN	80.4%	92.0%	97.0%
mean coverage	92.05%	94.84%	86.98%
fully covered (≥ 0.97)	19.5%	47.4%	0.8%
detected / missed	1,200 / 0	1,200 / 0	1,200 / 0
fragmentation (pred/GT)	2.00	1.66	2.04
pred segments	2,407	1,997	2,453
spurious (count / time)	25 / 40 s	15 / 15 s	1 / 1 s
over-extension (count / time)	1,252 / 1,151 s	111 / 77 s	8 / 5 s

Table 12: **Back Channel TC (Adapter-based VAD vs. LLM-based VAD vs. Silero VAD).**

Metric	Adapter-based	LLM-based	Silero
GT segments	1,200	1,200	1,200
TP	98.8%	99.0%	98.9%
FN	1.2%	1.0%	1.1%
FP	9.7%	9.0%	10.5%
TN	90.3%	91.0%	89.5%
mean coverage	99.03%	99.21%	99.16%
fully covered (≥ 0.97)	88.6%	91.3%	90.1%
detected / missed	1,200 / 0	1,200 / 0	1,200 / 0
fragmentation (pred/GT)	1.13	1.08	1.10
pred segments	1,702	1,573	1,716
spurious (count / time)	351 / 361 s	272 / 292 s	396 / 486 s
over-extension (count / time)	3 / 2 s	50 / 34 s	0 / 0 s

Back Channel TC—excluding back-channel voice. The key metric is spurious (Table 12). All three systems capture the true speech almost entirely (TP around 99%). Relative to the Adapter-based VAD, the cascade reduces spurious back-channel segments (351→272) while keeping coverage high. Relative to Silero, the cascade produces the fewest spurious segments of the three (272 vs. 396): the context-aware decision excludes back-channel sound that Silero captures as speech from its acoustic energy alone, and it does so with a comparable fully-covered rate for the true speech. The cascade shows a small amount of over-extension (50 sides / 34 s) that the Adapter-based VAD and Silero largely avoid, again reflecting its bias toward not terminating early.

Summary. Across the three test cases, the LLM-based cascade improves on the Adapter-based VAD by adding contextual verification—most visibly on Pause TC, where fully covered nearly doubles (19.5%→47.4%) and false positives and over-extension drop sharply—and improves on the purely acoustic Silero by maintaining utterances across pauses (far higher fully covered, lower fragmentation) and excluding non-speech such as back-channels (fewer spurious segments). The two heads play complementary roles: the adapter head contributes cheap, broad six-class discrimination and onset detection, while the LLM-based head contributes sharp, context-aware speech verification, and the cascade combines them. As a trade-off, the cascade tends to over-extend utterance endpoints slightly—a deliberate design choice to prevent premature termination—which can be tuned via the state-machine parameters.

5.4 Speech Decoder Evaluation

Within a real-time ALM, the LLM produces its response autoregressively, one text token at a time, and the Speech Decoder must process each token as it is emitted rather than waiting for the sentence or the full response to be completed. We therefore build the Speech Decoder on an architecture closely following that of Qwen3-TTS (Hu et al., 2026), which is designed for the same streaming condition: it takes text tokens as a streaming input and predicts a multi-token acoustic representation that is converted to a waveform by a neural audio codec decoder (Table 13). This lets the Speech Decoder attach directly to A.X K2 Light token stream. In this section, however, we detach it from the LLM and assess it purely as a

Table 13: Architectural specifications of compared speech decoders.

Model	# Params	Input streaming	Input type	Acoustic token type	Waveform generator
Fish-TTS	0.5B	×	Full text	Multi	Neural codec decoder
Qwen3-TTS	1.7B	✓	Text token	Multi	Neural codec decoder
Ours	0.88B	✓	Text token	Multi	Neural codec decoder
CosyVoice3	1.5B	✓	Text token	Single	Flow matching + vocoder

Table 14: Evaluation results of speech synthesis decoders. Bold marks the best performance among Ours, Fish-TTS, and Qwen3-TTS in each row.

Metric	Eval set	Ours (0.88B)	Fish-TTS (0.5B)	Qwen3-TTS (1.7B)	CosyVoice3* (1.5B)
CER (%) ↓	CV3-Eval-ko	4.79	4.30	4.00	18.76
	KSponSpeech-clean	4.35	8.44	7.06	120.34
	MiniMax-ko	3.38	2.87	2.93	8.47
UTMOS ↑	CV3-Eval-ko	3.374	3.011	3.400	3.764
	KSponSpeech-clean	3.547	2.997	3.280	3.546
	MiniMax-ko	3.360	3.001	3.415	3.802
UTMOSv2 ↑	CV3-Eval-ko	3.119	2.911	3.130	3.249
	KSponSpeech-clean	2.893	2.620	2.876	3.036
	MiniMax-ko	3.076	2.938	3.122	3.271
NISQA MOS ↑	CV3-Eval-ko	4.360	4.361	4.454	4.673
	KSponSpeech-clean	4.250	4.112	4.057	4.270
	MiniMax-ko	4.470	4.341	4.479	4.765
ScoreQ ↑	CV3-Eval-ko	3.812	3.588	3.797	4.108
	KSponSpeech-clean	3.907	3.313	3.652	3.944
	MiniMax-ko	3.852	3.552	3.843	4.103
STOI ↑	CV3-Eval-ko	0.992	0.989	0.992	0.993
	KSponSpeech-clean	0.988	0.979	0.985	0.986
	MiniMax-ko	0.993	0.988	0.991	0.993

*CosyVoice3 is included strictly as a performance reference and is excluded from direct model comparisons.

standalone TTS system, feeding it reference text rather than LLM output, in order to measure its intrinsic synthesis quality independently of the upstream model.

The evaluation is conducted on three Korean test sets: 600 sentences sampled from CV3-Eval-ko (Du et al., 2025), 3,000 from KSponSpeech-clean (Bang et al., 2020), and 100 from MiniMax-ko (MiniMax, 2025). The pronunciation accuracy is measured by the character error rate (CER), the perceptual quality by four learned MOS predictors—UTMOS (Saeki et al., 2022), UTMOSv2, NISQA MOS, and ScoreQ (Ragano et al., 2024)—and the intelligibility by SQUIM-based STOI (Taal et al., 2011). All metrics are reported per evaluation set.

We evaluate Speech Decoder against Fish-TTS, Qwen3-TTS, and CosyVoice3 (Du et al., 2025) (Table 13). For real-time ALM integration, minimizing response latency is critical, requiring a decoder capable of incrementally consuming text tokens as they are generated by the LLM. Fish-TTS requires a full input sentence before starting synthesis, introducing prohibitive delays that make it unsuitable for interactive deployment. CosyVoice3 accepts text tokens, but its single-token flow-matching backend relies on look-ahead blocks that impose inherent structural latency; furthermore, it suffers severe degradation when streaming Korean (CER 120.34% on KSponSpeech-clean due to reference-prompt leakage). We therefore treat CosyVoice3 strictly as a sound quality reference. Among the baselines, Qwen3-TTS offers the architecture best suited for real-time ALMs, pairing text-token streaming with a multi-token neural codec decoder for low-latency generation.

Adopting this streaming-friendly architecture, Speech Decoder achieves an favorable balance between model size, latency, and speech quality (Table 14). At just 0.88B parameters—roughly half the size of Qwen3-TTS (1.7B)—it delivers highly competitive perceptual quality: on CV3-Eval-ko and MiniMax-ko, the two models remain within a narrow margin of 0.05 UTMOS and 0.12 NISQA, while on the larger, more challenging KSponSpeech-clean set, Speech Decoder outperforms all baseline systems across every quality metric. Across all three test sets, Speech Decoder consistently leads in ScoreQ and equals or exceeds competing models in STOI, whereas Fish-TTS trails on nearly every quality measure. Regarding

pronunciation accuracy, Speech Decoder achieves the lowest CER on KSponSpeech-clean by a clear margin (4.35% vs. 7.06% for Qwen3-TTS and 8.44% for Fish-TTS) and stays close to the top performance on the two smaller sets (4.79% vs. 4.00% on CV3-Eval-ko; 3.38% vs. 2.87% on MiniMax-ko). Ultimately, Speech Decoder proves that a compact 0.88B decoder can match or surpass a model twice its size in perceptual quality while maintaining full text-token streaming for low-latency ALM pipelines.

5.5 System Performance

We evaluate the end-to-end latency on a single NVIDIA H200 GPU in a single-request setup, integrating the Adapter, A.X K2 Light, and Speech Decoder into a unified vLLM pipeline. The streaming encoder processes incoming user audio in real time, completing encoding as soon as the voice activity detector (VAD) signals the end of speech. Upon this hand-off, the accumulated embeddings are prefilled into the engine in a single batch, which marks the start of latency measurement. We evaluate system efficiency using two key metrics averaged over 100 utterances: *time-to-first-audio* (TTFA), defined as the delay from the VAD hand-off to the client receiving the first playable audio packet, and *real-time factor* (RTF), calculated as total generation time divided by output audio duration.

Table 15: **End-to-end latency on a single H200 GPU.** Results are averaged over 100 utterances in a single-request setup, measured from the VAD end-of-speech hand-off.

Metric	Average
Time-to-first-audio (TTFA)	≤ 500 ms
Real-time factor (RTF)	≤ 0.45

As shown in Table 15, our system responds in under 500 ms on average. This low latency is driven by fast execution across the entire pipeline—including the Adapter, A.X K2 Light, and Speech Decoder—together with a streaming encoder that processes audio while the user speaks. Because encoding overlaps with speech, TTFA reflects only prompt prefilling and first-chunk decoding. Furthermore, since all components share a single vLLM process, the system streams the first audio chunk before full text generation finishes, making TTFA dependent only on first-chunk latency rather than total response length. Finally, an RTF well below 1.0 shows that generation easily outpaces playback, leaving plenty of margin to handle network delays and buffering.

6 Limitations

While A.X K2 ALM shows that a frozen-LLM recipe can deliver competitive Korean speech capability, several limitations remain and shape our roadmap.

- **Residual ASR gap after dialogue training:** Speech-dialogue training (Stage B) introduces a modest regression in transcription accuracy relative to the Stage-A-aligned model (Sec. 5.1). Closing this gap—e.g., via multi-task or replay strategies during Stage B—is an active direction.
- **Knowledge-heavy spoken QA:** On knowledge-intensive multiple-choice subsets (BBH, Open-BookQA), A.X K2 ALM trails the largest baseline. This likely stems from backbone scale under the frozen-LLM constraint and from limited exposure to multiple-choice formats during training. Natural remedies include reinforcing the response format for knowledge queries and scaling the backbone within the A.X family, which our recipe supports without retraining the speech modules from scratch.
- **Language and modality scope:** The current model targets Korean and English, with the syllable-level alignment of Stage A exploiting the regular syllable–speech correspondence of Korean. Broader multilingual coverage and richer audio understanding (e.g., paralinguistics, environmental sounds, music) are left to future iterations.
- **Benchmark scope:** KVoiceBench currently evaluates single-turn queries with synthetic speech, under-representing real-speaker variation such as accents, disfluencies, and environmental noise. We plan to extend it with real speakers, noise conditions, and multi-turn dialogue.
- **VAD evaluation scope and endpoint latency:** The VAD is evaluated on a general recording set spanning clean and noisy environments together with two TTS-synthesized sets targeting mid-utterance pauses and back-channels; broader non-speech sound categories, finer-grained use of the six-class taxonomy, and validation on fully natural conversational speech from live deployment remain future work. The observed over-extension of utterance endpoints—a deliberate trade-off against premature termination—can be further tuned via the pause-wait and offset-confirmation parameters.

7 Conclusion

We present A.X K2 ALM, an audio language model that extends the frozen A.X K2 Light backbone (20B-A2.7B) to speech understanding, voice activity detection, and speech generation. Driven by the production requirement to add speech capabilities without compromising baseline text quality, our architecture freezes the core LLM and trains only dedicated speech-side components: a 300M chunk-wise Conformer encoder, an MLP + Q-Former adapter, dual VAD heads, and a multi-token speech decoder. This design delegates audio processing entirely to specialized modules, effectively leveraging the frozen LLM’s reasoning power without risking catastrophic forgetting or text performance degradation.

Our two-stage training aligns audio embeddings into the LLM representation space via the char2token bridge and cosine alignment (Stage A), and transfers response generation capabilities through front-weighted cross-entropy, logit distillation, and hidden-state alignment (Stage B). Across Korean benchmarks, A.X K2 ALM demonstrates strong performance across all speech modalities: its streaming ASR encoder outperforms Qwen3-Omni at half the parameter footprint; the context-aware VAD suppresses spurious back-channel triggers, handles mid-sentence pauses, and enables seamless barge-in; and the streaming speech decoder achieves superior pronunciation accuracy with high speech intelligibility comparable to leading systems. Overall, on KVoiceBench, A.X K2 ALM reaches 95% of Qwen3-Omni’s response quality at two-thirds of its size, while significantly outperforming Qwen2.5-Omni and HyperCLOVA X 8B Omni.

In future work, we plan to upgrade the neural audio codec to improve overall sound quality, close the residual ASR gap from dialogue tuning, and extend the framework toward fully end-to-end speech-to-speech interaction. Ultimately, A.X K2 ALM establishes a practical, scalable blueprint for equipping production LLMs with competitive, real-time voice capabilities.

8 Contributors

Contributors are listed in alphabetical order by last name, then first name.

Contributors

Donghoon Ahn, Jaecheol Bae, Taejun Bak, Seokjin Hong, Sungsoo Hwang, Changheon Lee, Donghyun Lee, Jaeguyn Lim, Minkyu Lim, ByungKwan Park, Jihun Park, Sunghwan Shin, and Juhong Yoo

Acknowledgement

We would like to thank the Foundation Model team at SK Telecom for providing the A.X K2 Light model, which serves as the LLM backbone of this work, and for their technical support throughout the project.

Appendix

A KVoiceBench: A Korean Spoken-Query Benchmark

The practical capability of an ALM cannot be characterized by transcription accuracy alone: a model can recover intent despite small transcription differences, or transcribe accurately and still fail on knowledge, reasoning, format, or safety. VoiceBench (Chen et al., 2024) evaluates this full path from audio perception to response generation, but its English-centric content does not transfer directly to Korean—constraints such as letter case are meaningless in Hangul, and regular-expression scoring can miss semantic equivalence in Korean answers. Existing Korean audio evaluations remain scarce (Kim et al., 2026). We therefore construct KVoiceBench, a Korean adaptation of VoiceBench that preserves the evaluated construct of each task while producing natural Korean items. KVoiceBench comprises ten evaluation sets covering general knowledge, open-ended helpfulness, instruction following, reasoning, and safety (Table 16); the current version evaluates single-turn Korean speech input with text output.

Table 16: **Composition of KVoiceBench.** Ten evaluation sets adapted from VoiceBench (Chen et al., 2024); MMSU is replaced by Korean-native professional-knowledge questions selected from KMMLU (Son et al., 2024). # Final denotes samples retained after the $\text{CER} \leq 3\%$ audio-quality filter (Appendix B). Open-ended sets are scored by an LLM judge (1–5, rescaled to 0–100); QA sets are scored by answer correctness.

Subset	Capability	Korean adaptation	# Source	# Final	Scoring
AdvBench	Safety	Translated & localized	520	464	LLM judge
AlpacaEval	Everyday helpfulness	Translated	199	158	LLM judge
AlpacaEval-Full	Open-ended knowledge	Translated (non-math/code)	636	522	LLM judge
CommonEval	Real-world queries	Translated	200	164	LLM judge
WildVoice	Diverse open-ended queries	Translated & localized	1,000	747	LLM judge
IFEval	Instruction/format following	Constraints rewritten	345	227	LLM judge
BBH	Logical reasoning	Translated & localized	1,000	790	Correctness
OpenBookQA	Science knowledge (MCQ)	Translated	455	402	Correctness
SD-QA	Reference-answer QA	Translated	553	367	Correctness
MMSU	Professional knowledge (MCQ)	Korean-native (KMMLU)	5,314	3,254	Correctness
Total			10,222	7,095	

Construction. Source items were first translated with LLMs and then human-reviewed for meaning, references, options, and output constraints; English-only constraints were rewritten as verifiable Korean constraints involving Hangul, punctuation, or style. To incorporate Korean-native professional knowledge and reduce translation bias, the MMSU subset is built not by translation but by selecting 5,314 questions from KMMLU (Son et al., 2024), preserving local institutions and terminology. Spoken queries were then synthesized with Google Chirp 3 (Google Cloud, 2025), randomly sampling one of 30 Korean voice styles per item with 0.5 s of silence padding at both ends (Fig. 9).

Audio quality control. Every synthesized waveform is re-transcribed with Whisper large-v3 (Radford et al., 2023) and retained only if the CER against the source text is at most 3%, ensuring that evaluation measures model capability rather than TTS artifacts. This filter retains 7,095 of 10,222 samples (69.4%). The threshold is empirically justified: with text input to an in-house LLM, the ten-set macro-average changes only from 86.55 (no filter) to 86.49 ($\text{CER} \leq 3\%$), indicating that the filter leaves task difficulty essentially intact, whereas an exact-match filter ($\text{CER} = 0$) would discard more than half of the samples and measurably shift subset difficulty. Per-subset retention statistics under different thresholds are provided in Appendix B.

Scoring protocol. All models receive the same filtered waveforms ($\text{CER} \leq 3\%$, 7,095 samples) and the common VoiceBench prompt, decoded at temperature 0.0 without explicit reasoning; the sole exception is HyperCLOVA X 8B Omni, which we evaluate with its own default system prompt, as it performs substantially better under this configuration than under the common prompt—we report the stronger setting and disclose the deviation here for transparency. Open-ended, safety, and instruction subsets are rated by an LLM judge (ChatGPT 5.4) (Zheng et al., 2023) on a 1–5 scale rescaled to 0–100; reference-QA and reasoning subsets are scored by semantic answer correctness, with judge output restricted to a single rating or Yes/No. The same judge version is used for all models. The overall score is the unweighted macro-average over the ten sets.

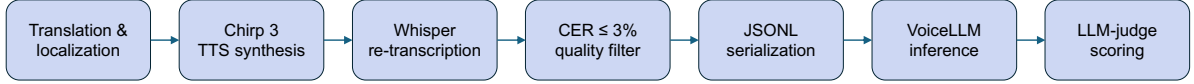


Figure 9: **KVoiceBench construction and evaluation pipeline.** Translated and localized items are synthesized with Chirp 3 TTS, validated by Whisper re-transcription with a $\text{CER} \leq 3\%$ filter, and evaluated through model inference and LLM-judge scoring.

B KVoiceBench Quality-Filtering Details

The construction pipeline of KVoiceBench is described in Appendix. A. For quality filtering, the Whisper transcript and the TTS source text are normalized (special characters removed, English uppercased) before computing $\text{CER} = (S + D + I) / N \times 100 (\%)$. Table 17 reports per-subset retention under CER thresholds of 5%, 3% (adopted), and 0%. The 3% threshold retains 69.4% of all samples while changing the text-input macro-average of an in-house LLM by only 0.06 points ($86.55 \rightarrow 86.49$); the exact-match filter ($\text{CER} = 0$) would retain only 48.5% of samples and measurably shift subset difficulty (e.g., BBH $69.10 \rightarrow 66.27$ under text input), motivating the adopted threshold.

Table 17: **KVoiceBench per-subset retention statistics.** Counts (and ratios relative to the original number of waveforms) of synthesized utterances whose Whisper transcription CER falls below each threshold. The 3% threshold is adopted for the final benchmark.

Subset	# Original	$\text{CER} \leq 5\%$	$\text{CER} \leq 3\%$ (adopted)	$\text{CER} = 0\%$
AdvBench	520	496 (95.4%)	464 (89.2%)	460 (88.5%)
AlpacaEval	199	176 (88.4%)	158 (79.4%)	146 (73.4%)
AlpacaEval-Full	636	558 (87.7%)	522 (82.1%)	482 (75.8%)
BBH	1,000	836 (83.6%)	790 (79.0%)	508 (50.8%)
CommonEval	200	169 (84.5%)	164 (82.0%)	163 (81.5%)
MMSU	5,314	4,046 (76.1%)	3,254 (61.2%)	1,820 (34.3%)
OpenBookQA	455	422 (92.8%)	402 (88.4%)	300 (65.9%)
SD-QA	553	386 (69.8%)	367 (66.4%)	367 (66.4%)
WildVoice	1,000	838 (83.8%)	747 (74.7%)	552 (55.2%)
IFEval	345	276 (80.0%)	227 (65.8%)	157 (45.5%)
Total	10,222	8,203 (80.3%)	7,095 (69.4%)	4,955 (48.5%)

C Details of Evaluation Benchmarks

Korean ASR. KsponSpeech (Bang et al., 2020) is a large-scale Korean spontaneous speech corpus; we evaluate on its eval-clean and eval-other splits. Zeroth-Korean (Jo & Lee, 2017) is an open Korean read-speech corpus. Both are scored with character error rate (CER), the standard metric for Korean ASR.

KVoiceBench subsets. Following VoiceBench (Chen et al., 2024), the subsets cover complementary aspects of voice-assistant capability: *AlpacaEval*, *AlpacaEval-Full*, *CommonEval*, and *WildVoice* evaluate open-ended instruction following and are scored by an LLM judge (OPEN, 1–5 rescaled to 0–100); *OpenBookQA* (Mihaylov et al., 2018) and *BBH* (Suzgun et al., 2023) evaluate knowledge and reasoning as multiple-choice QA (accuracy); *MMSU* evaluates professional knowledge with Korean-native questions selected from KMMLU (Son et al., 2024) (accuracy); *SD-QA* (Faisal et al., 2021) evaluates robustness of spoken QA; *IFEval* (Zhou et al., 2023) evaluates fine-grained instruction following; and *AdvBench* (Zou et al., 2023) evaluates safety refusal behavior.

TTS metrics. UTMOS (Saeki et al., 2022) is a neural MOS predictor for naturalness; STOI (Taal et al., 2011) measures intelligibility; PESQ (Rix et al., 2001) measures perceptual quality; ScoreQ (Ragano et al., 2024) is a model-based perceptual quality score. Pronunciation accuracy is measured as the CER of a strong ASR system on synthesized speech.

D Neural Audio Codec Evaluation

Setup. We evaluate the reconstruction fidelity of the SKT Audio Codec that underlies the Speech Decoder. We randomly select 2,000 sentences from a validation set not used in training; the original audio is encoded into discrete tokens by the Speech Tokenizer and then resynthesized into a waveform

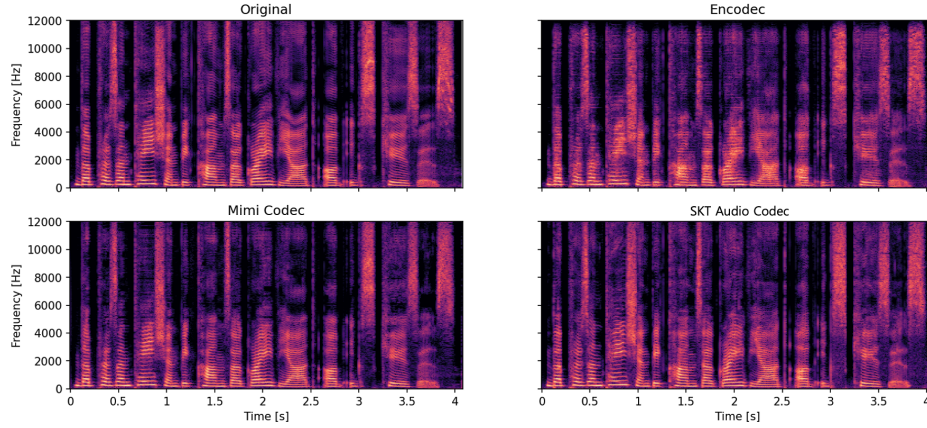


Figure 10: **Original vs. EnCodec, Mimi Codec, and SKT Audio Codec reconstruction spectrograms.** Mimi Codec and SKT preserve the original’s harmonic structure in the 0–4 kHz band almost identically, whereas EnCodec’s harmonic lines smear in the same band, losing detail. SKT still shows a slight loss relative to the original in the sharpness of high-frequency (>6 kHz) fricative/noise texture and onset transients.

by the Codec Decoder. We measure reference-based metrics that directly compare the reconstructed audio against the original—MCD (Mel-Cepstral Distortion), PESQ (Rix et al., 2001), STOI (Taal et al., 2011), F0 Corr (F0 tracking accuracy), ViSQOL, pysepm CD and LLR (segmental spectral distortion), and SpeechBERT, SpeechBLEU, and Token Distance (semantic/token similarity)—for the SKT Audio Codec described in this report, EnCodec (Défossez et al., 2022), and Mimi Codec (Défossez et al., 2024).

Table 18: **Compression Bitrate by Codec.**

Codec	Frame rate	Codebooks	Codebook size	Bits/codebook	Bitrate	Compression ratio	Size vs. original
EnCodec (6.0 kbps setting)	75 Hz	8	1024	10 bit	6,000 bps (6.0 kbps)	~64:1	1.56%
Mimi Codec	12.5 Hz	16	2048	11 bit	2,200 bps (2.2 kbps)	~174.5:1	0.57%
SKT Audio Codec	12.5 Hz	16	2048	11 bit	2,200 bps (2.2 kbps)	~174.5:1	0.57%

Compression efficiency. As summarized in Table 18, the SKT Audio Codec and Mimi Codec share the same Mimi-style configuration—a 12.5 Hz frame rate, a codebook size of 2,048, and all 16 codebooks used—and therefore operate at an identical 2.2 kbps, whereas EnCodec runs at 6.0 kbps, about $2.7\times$ higher. Despite this much lower bitrate, the spectrogram comparison in Figure 10 shows that EnCodec still loses more detail in the 2–4 kHz band than either SKT or Mimi, indicating that the SKT Audio Codec attains superior compression efficiency at a substantially lower bitrate.

Reconstruction quality. Table 19 reports the reference-based reconstruction metrics. Of the 10 metrics, the SKT Audio Codec ranks first on three—MCD, pysepm CD, and pysepm LLR—reflecting the strongest spectral-envelope match and the greatest suppression of segmental distortion; Mimi Codec leads on three (PESQ, STOI, F0 Corr) and EnCodec on the remaining four (ViSQOL, SpeechBERT, SpeechBLEU, Token Distance). This is consistent with the spectrograms in Figure 10, where Mimi Codec and SKT preserve the original’s harmonic structure and formant trajectories clearly in the low-to-mid band (0–4 kHz) while EnCodec smears the same band. In short, the SKT Audio Codec excels at precisely preserving the original’s overall spectral and prosodic shape, whereas perceptual quality (PESQ, STOI, ViSQOL) and semantic/token similarity (SpeechBERT, SpeechBLEU, Token Distance) remain relative strengths of Mimi Codec and EnCodec.

Table 19: **Audio Codec Quality Comparison.**

Method	MCD ↓	PESQ ↑	STOI ↑	F0 Corr ↑	ViSQOL ↑	pysepm CD ↓	pysepm LLR ↓	SpeechBERT ↑	SpeechBLEU ↑	Token Dist. ↑
EnCodec	6.148	2.603	0.943	0.605	4.190	3.695	0.490	0.938	0.678	0.818
Mimi Codec	6.632	2.776	0.945	0.629	3.909	3.699	0.470	0.916	0.649	0.796
SKT Audio Codec	5.729	2.259	0.923	0.612	4.171	3.335	0.367	0.921	0.669	0.814

E ALM Message Format

The conversation context is managed as a system / user / assistant message history, where the user content contains the embedding vector generated by the Encoder with VAD. The message used for an Audio LLM request takes the following form, where the user content consists of a Base64-encoded audio embedding (audio_embeds) and a text part.

```
[
  { "role": "system",
    "content": "You are an AI assistant that responds naturally and warmly." },
  { "role": "user",
    "content": [
      { "type": "audio_embeds", "audio_embeds": "<Base64 Encoded Embedding Vector>" },
      { "type": "text", "text": "" }
    ] },
  { "role": "assistant", "content": "Sure, hello." },
  { "role": "user",
    "content": [
      { "type": "audio_embeds", "audio_embeds": "<Base64 Encoded Embedding Vector>" },
      { "type": "text", "text": "" }
    ] }
]
```

References

- Josh Achiam, Steven Adler, Sandhini Agarwal, Lama Ahmad, Ilge Akkaya, Florencia Leoni Aleman, Diogo Almeida, Janko Altschmidt, Sam Altman, Shyamal Anadkat, et al. GPT-4 technical report. *arXiv preprint arXiv:2303.08774*, 2023.
- Jean-Baptiste Alayrac, Jeff Donahue, Pauline Luc, Antoine Miech, Iain Barr, Yana Hasson, Karel Lenc, Arthur Mensch, Katherine Millican, Malcolm Reynolds, et al. Flamingo: a visual language model for few-shot learning. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2022.
- Alexei Baevski, Yuhao Zhou, Abdelrahman Mohamed, and Michael Auli. wav2vec 2.0: A framework for self-supervised learning of speech representations. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2020.
- Jeong-Uk Bang, Seung Yun, Seung-Hi Kim, Mu-Yeol Choi, Min-Kyu Lee, Yeo-Jeong Kim, Dong-Hyun Kim, Jun Park, Young-Jik Lee, and Sang-Hun Kim. KsponSpeech: Korean spontaneous speech corpus for automatic speech recognition. *Applied Sciences*, 10(19):6936, 2020.
- Ilja Baumann, Dominik Wagner, Korbinian Riedhammer, and Tobias Bocklet. Optimized self-supervised training with BEST-RQ for speech recognition. In *IEEE International Conference on Acoustics, Speech, and Signal Processing (ICASSP)*, 2025. arXiv:2501.16131.
- Sanyuan Chen, Chengyi Wang, Zhengyang Chen, Yu Wu, Shujie Liu, Zhuo Chen, Jinyu Li, Naoyuki Kanda, Takuya Yoshioka, Xiong Xiao, et al. WavLM: Large-scale self-supervised pre-training for full stack speech processing. *IEEE Journal of Selected Topics in Signal Processing*, 16(6):1505–1518, 2022.
- Yiming Chen, Xianghu Yue, Chen Zhang, Xiaoxue Gao, Robby T. Tan, and Haizhou Li. VoiceBench: Benchmarking LLM-based voice assistants. *arXiv preprint arXiv:2410.17196*, 2024.
- Chung-Cheng Chiu, James Qin, Yu Zhang, Jiahui Yu, and Yonghui Wu. Self-supervised learning with random-projection quantizer for speech recognition. In *International Conference on Machine Learning (ICML)*, pp. 3915–3924, 2022.
- Yunfei Chu, Jin Xu, Qian Yang, Haojie Wei, Xipin Wei, Zhifang Guo, Yichong Leng, Yuanjun Lv, Jinzheng He, Junyang Lin, Chang Zhou, and Jingren Zhou. Qwen2-Audio technical report. *arXiv preprint arXiv:2407.10759*, 2024.
- Gheorghe Comanici, Eric Bieber, Mike Schaekermann, Ice Pasupat, Noveen Sachdeva, Inderjit Dhillon, Marcel Blistein, Ori Ram, Dan Zhang, Evan Rosen, et al. Gemini 2.5: Pushing the frontier with advanced reasoning, multimodality, long context, and next generation agentic capabilities. *arXiv preprint arXiv:2507.06261*, 2025.
- Alexandre Défossez, Jade Copet, Gabriel Synnaeve, and Yossi Adi. High fidelity neural audio compression. *arXiv preprint arXiv:2210.13438*, 2022.
- Alexandre Défossez, Laurent Mazaré, Manu Orsini, Amélie Royer, Patrick Pérez, Hervé Jégou, Edouard Grave, and Neil Zeghidour. Moshi: a speech-text foundation model for real-time dialogue. *arXiv preprint arXiv:2410.00037*, 2024.
- Brecht Desplanques, Jenthe Thienpondt, and Kris Demuynck. ECAPA-TDNN: Emphasized channel attention, propagation and aggregation in TDNN based speaker verification. In *Proc. Interspeech*, pp. 3830–3834, 2020.
- Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. BERT: Pre-training of deep bidirectional transformers for language understanding. In *Proceedings of NAACL-HLT*, pp. 4171–4186, 2019.
- Zhihao Du, Changfeng Gao, Yuxuan Wang, Fan Yu, Tianyu Zhao, Hao Wang, Xiang Lv, Hui Wang, Xian Shi, Keyu An, et al. CosyVoice 3: Towards in-the-wild speech generation via scaling-up and post-training. *arXiv preprint arXiv:2505.17589*, 2025.
- ElevenLabs. ElevenLabs text-to-speech, 2025. URL <https://elevenlabs.io>.
- Fahim Faisal, Sharlina Keshava, Md Mahfuz ibn Alam, and Antonios Anastasopoulos. SD-QA: Spoken dialectal question answering for the real world. In *Findings of the Association for Computational Linguistics: EMNLP 2021*, pp. 3296–3315, 2021.
- Ruchao Fan, Bhiksha Shankar, et al. AlignFormer: Modality matching can achieve better zero-shot instruction-following speech-LLM. *IEEE Journal of Selected Topics in Signal Processing*, 2025. arXiv:2412.01145.

-
- Qingkai Fang, Shoutao Guo, Yan Zhou, Zhengrui Ma, Shaolei Zhang, and Yang Feng. LLaMA-Omni: Seamless speech interaction with large language models. *arXiv preprint arXiv:2409.06666*, 2024.
- Yassir Fathullah, Chunyang Wu, Egor Lakomkin, Ke Li, Junteng Jia, Yuan Shangguan, Jay Mahadeokar, Ozlem Kalinli, Christian Fuegen, and Mike Seltzer. AudioChatLlama: Towards general-purpose speech abilities for LLMs. In *Proceedings of NAACL*, 2024.
- Google Cloud. Google cloud text-to-speech API, 2025. URL <https://cloud.google.com/text-to-speech>.
- Alex Graves. Sequence transduction with recurrent neural networks. *arXiv preprint arXiv:1211.3711*, 2012.
- Alex Graves, Santiago Fernández, Faustino Gomez, and Jürgen Schmidhuber. Connectionist temporal classification: Labelling unsegmented sequence data with recurrent neural networks. In *International Conference on Machine Learning (ICML)*, pp. 369–376, 2006.
- Anmol Gulati, James Qin, Chung-Cheng Chiu, Niki Parmar, Yu Zhang, Jiahui Yu, Wei Han, Shibo Wang, Zhengdong Zhang, Yonghui Wu, and Ruoming Pang. Conformer: Convolution-augmented transformer for speech recognition. In *Proc. Interspeech*, pp. 5036–5040, 2020.
- Geoffrey Hinton, Oriol Vinyals, and Jeff Dean. Distilling the knowledge in a neural network. *arXiv preprint arXiv:1503.02531*, 2015.
- Wei-Ning Hsu, Benjamin Bolte, Yao-Hung Hubert Tsai, Kushal Lakhotia, Ruslan Salakhutdinov, and Abdelrahman Mohamed. HuBERT: Self-supervised speech representation learning by masked prediction of hidden units. *IEEE/ACM Transactions on Audio, Speech, and Language Processing*, 29:3451–3460, 2021.
- Hangrui Hu et al. Qwen3-TTS technical report. *arXiv preprint arXiv:2601.15621*, 2026.
- Won Jang, Dan Lim, Jaesam Yoon, Bongwan Kim, and Juntae Kim. Univnet: A neural vocoder with multi-resolution spectrogram discriminators for high-fidelity waveform generation. *arXiv preprint arXiv:2106.07889*, 2021.
- Lucas Jo and Wonkyum Lee. Zeroth-korean: Korean open-source speech corpus, 2017. URL <https://github.com/goodatlas/zeroth>.
- Wonjune Kang et al. Frozen large language models can perceive paralinguistic aspects of speech. In *Proc. Interspeech*, 2025. *arXiv:2410.01162*.
- Jinyoung Kim et al. KoALa-Bench: Evaluating large audio language models on korean speech understanding and faithfulness. *arXiv preprint arXiv:2604.19782*, 2026.
- Kimi Team. Kimi-Audio technical report. *arXiv preprint arXiv:2504.18425*, 2025.
- KRAFTON AI. Raon speech technical report, 2026. Technical report.
- Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E. Gonzalez, Hao Zhang, and Ion Stoica. Efficient memory management for large language model serving with PagedAttention. In *Proceedings of the 29th Symposium on Operating Systems Principles (SOSP)*, pp. 611–626, 2023.
- Junnan Li, Dongxu Li, Silvio Savarese, and Steven Hoi. BLIP-2: Bootstrapping language-image pre-training with frozen image encoders and large language models. In *International Conference on Machine Learning (ICML)*, pp. 19730–19742, 2023.
- Ziyang Ma, Yakun Song, Chenpeng Du, Jian Cong, Zhuo Chen, Yuping Wang, Yuxuan Wang, and Xie Chen. Language model can listen while speaking. *arXiv preprint arXiv:2408.02622*, 2024.
- Todor Mihaylov, Peter Clark, Tushar Khot, and Ashish Sabharwal. Can a suit of armor conduct electricity? a new dataset for open book question answering. In *Proceedings of EMNLP*, pp. 2381–2391, 2018.
- MiniMax. MiniMax-Speech: Intrinsic zero-shot text-to-speech with a learnable speaker encoder. *arXiv preprint arXiv:2505.07916*, 2025.
- NAVER Cloud HyperCLOVA X Team. HyperCLOVA X 8B Omni. *arXiv preprint arXiv:2601.01792*, 2026.
- OpenAI. GPT-4o system card. *arXiv preprint arXiv:2410.21276*, 2024.
- Daniel S. Park, William Chan, Yu Zhang, Chung-Cheng Chiu, Barret Zoph, Ekin D. Cubuk, and Quoc V. Le. SpecAugment: A simple data augmentation method for automatic speech recognition. In *Proc. Interspeech*, pp. 2613–2617, 2019.

-
- Jing Peng et al. TASU: Text-only alignment for speech understanding. In *IEEE International Conference on Acoustics, Speech, and Signal Processing (ICASSP)*, 2026. arXiv:2511.03310.
- Matthew E. Peters, Mark Neumann, Mohit Iyyer, Matt Gardner, Christopher Clark, Kenton Lee, and Luke Zettlemoyer. Deep contextualized word representations. In *Proceedings of NAACL-HLT*, pp. 2227–2237, 2018.
- Qwen Team. Qwen3-Omni technical report. *arXiv preprint arXiv:2509.17765*, 2025.
- Alec Radford, Jong Wook Kim, Tao Xu, Greg Brockman, Christine McLeavey, and Ilya Sutskever. Robust speech recognition via large-scale weak supervision. In *International Conference on Machine Learning (ICML)*, pp. 28492–28518, 2023.
- Alessandro Ragano, Jan Skoglund, and Andrew Hines. SCOREQ: Speech quality assessment with contrastive regression. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2024.
- Antony W. Rix, John G. Beerends, Michael P. Hollier, and Andries P. Hekstra. Perceptual evaluation of speech quality (PESQ)-a new method for speech quality assessment of telephone networks and codecs. In *IEEE International Conference on Acoustics, Speech, and Signal Processing (ICASSP)*, volume 2, pp. 749–752, 2001.
- Takaaki Saeki, Detai Xin, Wataru Nakata, Tomoki Koriyama, Shinnosuke Takamichi, and Hiroshi Saruwatari. UTMOS: UTokyo-SaruLab system for VoiceMOS challenge 2022. In *Proc. Interspeech*, pp. 4521–4525, 2022.
- Silero Team. Silero VAD: pre-trained enterprise-grade voice activity detector, 2021. URL <https://github.com/snakers4/silero-vad>.
- SK Telecom. A.X K1 technical report. *arXiv preprint arXiv:2601.09200*, 2026.
- Guijin Son, Hanwool Lee, Sungdong Kim, Seungone Kim, Niklas Muennighoff, Taekyoon Choi, Cheonbok Park, Kang Min Yoo, and Stella Biderman. KMMLU: Measuring massive multitask language understanding in korean. *arXiv preprint arXiv:2402.11548*, 2024.
- Mirac Suzgun, Nathan Scales, Nathanael Schärli, Sebastian Gehrmann, Yi Tay, Hyung Won Chung, Aakanksha Chowdhery, Quoc Le, Ed Chi, Denny Zhou, et al. Challenging BIG-Bench tasks and whether chain-of-thought can solve them. In *Findings of the Association for Computational Linguistics: ACL 2023*, pp. 13003–13051, 2023.
- Cees H. Taal, Richard C. Hendriks, Richard Heusdens, and Jesper Jensen. An algorithm for intelligibility prediction of time-frequency weighted noisy speech. *IEEE Transactions on Audio, Speech, and Language Processing*, 19(7):2125–2136, 2011.
- Changli Tang, Wenyi Yu, Guangzhi Sun, Xianzhao Chen, Tian Tan, Wei Li, Lu Lu, Zejun Ma, and Chao Zhang. SALMONN: Towards generic hearing abilities for large language models. In *International Conference on Learning Representations (ICLR)*, 2024.
- Aaron Van Den Oord, Oriol Vinyals, et al. Neural discrete representation learning. *Advances in neural information processing systems*, 30, 2017.
- Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2017.
- Chen Wang, Minpeng Liao, Zhongqiang Huang, Jinliang Lu, Junhong Wu, Yuchen Liu, Chengqing Zong, and Jiajun Zhang. BLSP: Bootstrapping language-speech pre-training via behavior alignment of continuation writing. *arXiv preprint arXiv:2309.00916*, 2023.
- Feng Wang, Jian Cheng, Weiyang Liu, and Haijun Liu. Additive margin softmax for face verification. *IEEE Signal Processing Letters*, 25(7):926–930, 2018.
- Xiong Wang, Yangze Li, Chaoyou Fu, Yunhang Shen, Lei Xie, Ke Li, Xing Sun, and Long Ma. Freeze-omni: A smart and low latency speech-to-speech dialogue model with frozen LLM. *arXiv preprint arXiv:2411.00774*, 2024.
- Jin Xu, Zhifang Guo, Jinzheng He, Hangrui Hu, Ting He, Shuai Bai, Keqin Chen, Jialin Wang, Yang Fan, Kai Dang, Bin Zhang, Xiong Wang, Yunfei Chu, and Junyang Lin. Qwen2.5-Omni technical report. *arXiv preprint arXiv:2503.20215*, 2025.

-
- Biao Zhang and Rico Sennrich. Root mean square layer normalization. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2019.
- Lianmin Zheng, Wei-Lin Chiang, Ying Sheng, Siyuan Zhuang, Zhanghao Wu, Yonghao Zhuang, Zi Lin, Zhuohan Li, Dacheng Li, Eric P. Xing, Hao Zhang, Joseph E. Gonzalez, and Ion Stoica. Judging LLM-as-a-judge with MT-Bench and chatbot arena, 2023.
- Jeffrey Zhou, Tianjian Lu, Swaroop Mishra, Siddhartha Brahma, Sujoy Basu, Yi Luan, Denny Zhou, and Le Hou. Instruction-following evaluation for large language models, 2023.
- Andy Zou, Zifan Wang, Nicholas Carlini, Milad Nasr, J. Zico Kolter, and Matt Fredrikson. Universal and transferable adversarial attacks on aligned language models. *arXiv preprint arXiv:2307.15043*, 2023.